

Understanding Open Source Contributor Profiles in Popular Machine Learning Libraries

JIAWEN LIU, Department of Electrical and Computer Engineering, Queen's University, Canada

HAOXIANG ZHANG, Software Analysis and Intelligence Lab (SAIL), Queen's University, Canada

YING ZOU, Department of Electrical and Computer Engineering, Queen's University, Canada

With the increasing popularity of machine learning (ML), many open source software (OSS) contributors are attracted to developing and adopting ML approaches. Comprehensive understanding of ML contributors is crucial for successful ML OSS development and maintenance. Without such knowledge, there is a risk of inefficient resource allocation and hindered collaboration in ML OSS projects. Existing research focuses on understanding the difficulties and challenges perceived by ML contributors through user surveys. There is a lack of understanding of ML contributors based on their activities recorded in the software repositories. In this paper, we aim to understand ML contributors by identifying contributor profiles in ML libraries. We further study contributors' OSS engagement from four aspects: workload composition, work preferences, technical importance, and ML-specific vs SE contributions. By investigating 11,949 contributors from 8 popular ML libraries (i.e., TensorFlow, PyTorch, scikit-learn, Keras, MXNet, Theano/Aesara, ONNX, and deeplearning4j), we categorize them into four contributor profiles: *Core-Nighttime*, *Core-Daytime*, *Peripheral-Nighttime*, and *Peripheral-Daytime*. We find that: 1) project experience, authored files, collaborations, pull requests comments received and approval ratio, and geographical location are significant features of all profiles; 2) contributors in Core profiles exhibit significantly different OSS engagement compared to Peripheral profiles; 3) contributors' work preferences and workload compositions are significantly correlated with project popularity; and 4) long-term contributors evolve towards making fewer, constant, balanced and less technical contributions.

CCS Concepts: • **Software and its engineering** → **Programming teams**.

Additional Key Words and Phrases: Open Source Software, Developer Profiles, Collaborative Software Development, Machine Learning Libraries

ACM Reference Format:

Jiawen Liu, Haoxiang Zhang, and Ying Zou. 2025. Understanding Open Source Contributor Profiles in Popular Machine Learning Libraries. 1, 1 (July 2025), 59 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Open Source Software (OSS) has emerged as a dominant model in software development, gaining widespread recognition among enterprises and developers as the preferred approach for software development. The OSS community comprises globally distributed contributors and users with shared interests, who actively participate in knowledge sharing and collaborate on the development and maintenance of software projects. Anyone with the necessary knowledge and

Authors' addresses: Jiawen Liu, Department of Electrical and Computer Engineering, Queen's University, Kingston, Canada, jiawen.liu@queensu.ca; Haoxiang Zhang, Software Analysis and Intelligence Lab (SAIL), Queen's University, Kingston, Canada; Ying Zou, Department of Electrical and Computer Engineering, Queen's University, Kingston, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

skills can be a contributor to OSS projects. They can contribute in various ways, such as writing source code, updating documentation, reporting issues, conducting code reviews, and participating in issue discussions. These activities are critical to the success of OSS projects. The rising popularity of open-source ML libraries, such as Tensorflow¹ and PyTorch², has facilitated ML implementation, attracting a growing number of software developers to learn ML technologies and contribute to ML projects [34].

Developers are the most important resource in software development and maintenance. A deep understanding of developer team composition could provide valuable insights for software development and management. This is particularly important in OSS projects, as contributors are distributed around the globe and have diverse backgrounds. Therefore, it is challenging for project managers and maintainers to have comprehensive knowledge to facilitate contributor productivity. Moreover, ML is heavily integrated into software services affecting people’s daily lives. ML related software components are implemented by leveraging the popular open-source ML libraries such as Tensorflow and Pytorch. Such open source ML libraries become critical to the performance and reliability of the software built on top of them. A large number of contributors are actively involved in the development of open-source ML libraries (e.g., around 3,500 contributors for Tensorflow), and these contributors’ behaviors, work patterns, and progressions within the ML projects are important to the success of both the open-source ML projects themselves and the downstream software services that depend on them.

Extensive research has been conducted to portray or categorize OSS contributors from various individual perspectives, such as the volume of contribution [22, 64, 65], technical expertise [24, 66], social networks [18, 27], duration [7, 109], and contribution dynamics [103], particularly in traditional software projects that are not specific to ML. However, few have endeavored to provide a comprehensive and multi-facet understanding of OSS contributors considering their behaviors, expertise, workload, work preferences, and the importance of their contributions. A lack of comprehensive insight may lead to ineffective resource allocation or collaborative challenges. For example, as reported by Balali et al. [5], contributors face challenges in their collaborations such as communication issues caused by timezone, language and cultural differences, difficulties in managing time to collaborate, mismatch of knowledge background, and harsh project atmosphere.

To investigate the challenges faced by ML developers, existing studies primarily conduct user surveys and interviews to collect insights into developer experiences, demands, and pain points, and they mainly report the challenges that developers encounter while learning or working with ML [11, 43, 46]. In terms of the quantitative study, Han et al. [38] have conducted an empirical study on the onboarding process for newcomers to deep learning projects. However, there remains a lack of quantitative studies to comprehensively portray ML contributors and their characteristics throughout their entire course of contribution. Moreover, survey-based findings are often constrained by the limited number of participants and the subjective nature of self-reported data.

To enhance our understanding of ML contributors, we conduct a large-scale and comprehensive study by mining the software repositories of 8 popular ML libraries (i.e., TensorFlow, PyTorch, scikit-learn, Keras, MXNet, Theano/Aesara, ONNX, and deeplearning4j), to obtain complete historical records of contributor activities. Specifically, we analyze a total of 11,949 contributors with 287,634 commits, 146,950 pull requests, and 114,912 issues from these ML libraries, to study ML contributors’ profiles, various OSS engagements, and evolutions over time. Our findings provide a multi-facet view of ML contributors and can be complementary to prior qualitative studies and overcome the limitations in scale and subjectivity. In addition to the previous studies that identify the challenges faced by ML developers, we aim to

¹<https://www.tensorflow.org/>

²<https://pytorch.org/>

learn from the behaviors and activities of experienced ML contributors in these established ML libraries and provide solutions and suggestions to help ML OSS contributors progress and improve ML project management. Particularly, we aim to address the following research questions:

RQ1: What are the characteristics of contributor profiles? We categorize developers into four distinct contributor profiles (i.e., *Core-Nighttime*, *Core-Daytime*, *Peripheral-Nighttime*, and *Peripheral-Daytime*) based on their contributing times, number of commits, code contribution density, and programming language usage. To gain a further understanding of these profiles, we build four binary logistic regression models to analyze the important features associated with each profile. We find that project experience, authored files, average pull request comments received, pull request approval ratio, and collaborations are significant for all profiles, mainly distinguishing *Core* from *Peripheral* profiles.

RQ2: What is the OSS engagement of each contributor profile? We observe in RQ1 that contributors within the same profile have different OSS engagements. To further study the OSS engagement by contributors of different profiles, we analyze contributors from four aspects: **workload composition** that describes a contributor's focus of work on the five key OSS activities within a period (i.e., reporting issues, issue discussions, commits, pull request discussions, and code reviews); **work preferences** that capture the dynamics and balance of a contributor's contributions within a period; **technical importance** that measures the importance of a contributor's contribution; and **ML-specific vs SE contributions** which categorize the content of contributions into 5 ML components [13] (i.e., User-Level API, Graph-Level Implementation, Operation Implementation, General Utility, and Environment-Dependent Processing), versus general software development. We identify five workload composition patterns that capture the contributors' common workload compositions, extract nine work preference features, introduce four technical importance metrics, and label 384 pull requests in terms of ML components. We apply the Chi-square test to compare the distribution of contributors with different workload compositions and the distribution of contributions to different ML components across profiles. We apply the Mann-Whitney U test to compare contributors' work preferences and technical importance across profiles. We observe significant differences in the workload composition, work preference, technical importance, and ML-specific contributions between *Core* and *Peripheral* contributors.

RQ3: Which factors of contributor OSS engagement are related to a project's popularity growth? To understand the impact of contributor OSS engagements on the project, we investigate the correlation between the distribution of contributor workload composition and work preferences with the growth of project popularity, measured by star ratings and forks. We build four mixed-effect models to study the relationship between the distribution of contributor's work preferences and workload compositions with the increase of stars and forks respectively. We find that work preference towards balanced contributions and conducting more code reviews are significantly correlated with the increase in star ratings and forks. Additionally, a higher presence of Issue Reporters and Issue Discussants in the project and a higher portion of code reviews made by PR Discussants are also correlated with the increase in stars and forks.

RQ4: How does contributor OSS engagement evolve? We observe in RQ2 that contributors' OSS engagement changes over time. To understand the evolution of OSS engagement, we apply the Cox-Stuart trend test to examine the significant trends in workload composition, work preference, and technical importance in contributors' active periods. We observe a portion of long-term contributors evolve towards fewer, constant, and balanced contribution dynamics, and shift away from highly technical and intensive contributions.

In summary, we make the following contributions to the software engineering community:

(1) We provide a comprehensive understanding of ML contributors from both static and dynamic points of view by studying their activities in a period and analyzing their engagement over time.

(2) Through both quantitatively and qualitatively studying four contributor profile categories in eight popular ML libraries, we establish a thorough understanding of ML contributors based on their OSS activities and provide actionable suggestions to support their progression and ML OSS project management.

(3) We identify how contributor work preferences and workload compositions are correlated with the growth of project popularity.

(4) We observe the distribution difference of ML-specific contributions between *Core* and *Peripheral* contributors, providing insights into how a contributor can progress in terms of ML-specific expertise and code contribution.

Paper organization. The remainder of our paper is organized as follows. Section 2 describes data collection and feature extraction. Section 3 describes how we create contributor profiles. Section 4 explains our approach for identifying four types of contributor OSS engagements and the resulting patterns. Section 5 presents the motivations, approaches, and results of our research questions. Section 6 extends and discusses our results to provide suggestions to ML contributors. We describe the implications in Section 7 and threats to the validity of our study in Section 8. Section 9 describes relevant studies. Lastly, we conclude our paper in Section 10 and provide our replication package in Section 11.

2 Experiment Setup

In this section, we present data collection and experiment setup. An overview of our approach is shown in Figure 1. We first collect the subject projects and their historical data from GitHub. Preprocessing is conducted on the collected project data, which involves removing non-human contributors and cleaning noises that may affect contributor feature calculation. Then, we extract ML contributors within the subject projects and extract contributor features from the processed data. Lastly, we identify contributor profiles, workload composition patterns, work preference features, technical importance, and manually analyze ML-specific contributions.

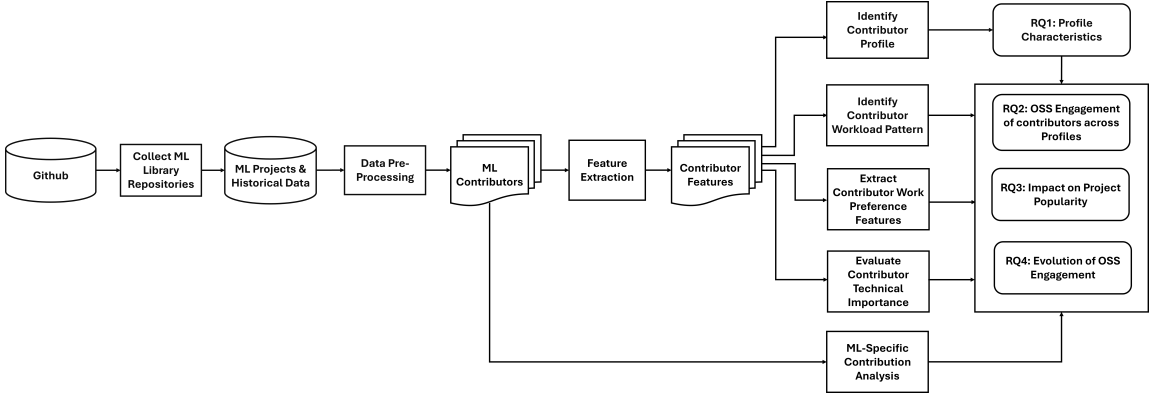


Fig. 1. The overview of our approach.

2.1 Data Collection

We select the subject projects and collect the history data from Github using Github REST API [1] and PyDriller [88].

Project Selection. To conduct our experiments on projects that contain contributor information related to machine learning technologies, we first review existing works that have studied popular machine learning libraries

Table 1. Descriptive statistics of the selected ML library projects.

Project Name	Contributors	Commits	Pull Requests	Issues	Creation Time
Tensorflow ⁴	3,499	147,242	23,518	37,020	2015-11
PyTorch ⁵	3,085	60,834	77,491	39,364	2012-01
scikit-learn ⁶	2,647	27,880	16,125	9,160	2010-01
Keras ⁷	1,139	9,967	6,843	11,606	2015-03
MXNet ⁸	836	11,535	10,919	7,763	2015-04
Theano/Aesara ^{9,10}	403	24,990	4,713	2,497	2008-01
ONNX ¹¹	274	2,505	3,035	2,387	2012-01
DeepLearning4J ¹²	66	2,681	4,306	5,115	2019-06

and frameworks for various purposes [10, 25, 28, 30–33, 35–40, 45, 47, 49–51, 53–56, 58–60, 63, 69, 75, 79, 86, 90–92, 94, 95, 98, 100, 105, 108, 110] (details can be found in our online supplementary material³). From this literature review, we select representative ML libraries that are widely used and commonly studied within the software engineering community.

Based on their frequency in the reviewed literature, we identify the following ten popular ML libraries: TensorFlow (36 mentions), PyTorch (32), Keras (25), Caffe/Caffe2 (18), Theano/Aesara (16), Scikit-learn (12), MXNet (11), CNTK (8), Chainer (6), and deeplearning4j (4). However, we exclude Chainer, CNTK, Caffe, and Caffe2 as their GitHub repositories have not been actively maintained or updated since 2019 or 2020. Although Theano is no longer under development after 2017, its continued project, Aesara, is active at the year of our data collection (i.e., 2023). Therefore, we consider Theano and Aesara as the same project and keep them in our subject projects. We include ONNX in our subject projects despite its limited presence (once) in the literature we explored, as it is an emerging library crucial for ML deployments. Its significance is widely recognized, and prior studies, such as [106], have highlighted it as an area for future research. Thus, our final selection consists of eight open-source ML libraries: TensorFlow, PyTorch, Keras, Theano/Aesara, Scikit-learn, MXNet, deeplearning4j, and ONNX.

All the selected projects contain more than 2k commits and have been active for more than 5 years, ensuring that they provide sufficient information for our analysis. A detailed overview of the subject projects is presented in Table 1.

Collecting Data. After the subject projects are selected, we retrieve the project development data from their repositories on Github. We use Github REST API¹³ to collect (i) commits; (ii) pull requests; (iii) issue reports of each project, and (iv) the account type of contributors (e.g., bot, organization, or user account) in the projects. PyDriller [88] is used to fetch the commit timestamp in the local time of the commit authors and their time zones.

2.2 Data Preprocessing

2.2.1 Removing Nonhuman Contributors

³https://github.com/Software-Evolution-Analytics-Lab-SEAL/ML_Contributor_replication/blob/main/project_selection_literatures.pdf

⁴<https://github.com/tensorflow/tensorflow>

⁵<https://github.com/pytorch/pytorch>

⁶<https://github.com/scikit-learn/scikit-learn>

⁷<https://github.com/keras-team/keras>

⁸<https://github.com/apache/mxnet>

⁹<https://github.com/Theano/Theano>

¹⁰<https://github.com/aesara-devs/aesara>

¹¹<https://github.com/onnx/onnx>

¹²<https://github.com/deeplearning4j/deeplearning4j>

¹³<https://docs.github.com/en/rest?apiVersion=2022-11-28>

We observe that bots are employed in our subject projects for various project management tasks, such as synchronizing changes from the public repository on GitHub to an internal repository (e.g., tensorflow-gardener¹⁴ and facebook-github-bot¹⁵), labeling issues (e.g., mxnet-label-bot¹⁶), merging pull requests (e.g., pytorchmergebot¹⁷), or testing changes (e.g., tensorflow-jenkins). As the focus of this study is to understand human contributors, we apply a rigorous method to filter out bots. However, as AI becomes a promising driving force for software development, future work can further study bots to analyze their behaviors and how they cooperate with human contributors in modern OSS development.

To filter out bots, we first extract all the usernames of contributors who make at least one commit in the selected ML projects. We then use the GitHub Users API¹⁸ to check if each username belongs to a user, bot, organization, or enterprise type account and only retain those with user accounts. Next, we sort the contributors in each project by their total number of commits and manually investigate the remaining contributors to identify any contributors with abnormal usernames (e.g., contain 'bot' or contain similar characters as the project name) or behaviors (e.g., making a large number of commits without raising any pull requests or making comments). For each identified contributor, we check their Github profile, examine if it is mentioned in the project ReadMe or Wiki, and search for any public announcements or discussions by the development team to confirm its use as a bot. As a result, we identify and remove 14 bots (i.e., tensorflow-gardener, tensorflow-jenkins, onnxbot, facebook-github-bot, pytorchmergebot, pytorchbot, docusaurus-bot, theano-bot, deadsnakes-issues-bot, mxnet-label-bot, snyk-bot, pytorchupdatebot, scikit-learn-bot, and inclusive-coding-bot). Then, we use the GitHub bot detection tool, BotHunter, proposed by Abdellatif et al.[2] to validate whether the remaining contributors are human. We manually examine the GitHub profiles of 54 users identified as bots by BotHunter and observe that 53 identified bots are false positives (i.e., they are not bots) and BotHunter identified one bot correctly (i.e., sklearn-ci).

2.2.2 Cleaning Noises in Pull Request Data

To extract contributor features related to pull request acceptance, such as the number of merged pull requests and the pull request approval ratio for a contributor, we need to determine whether a pull request was accepted. We use the pull request close status on Github to determine its acceptance, where accepted pull requests are set to "Merged" status and the rejected pull requests are set to "Closed". However, we observe that PyTorch uses different strategies to indicate PR approval over time. Before July 2018, accepted pull requests in Pytorch are set to "Merged" status. From May 2019 onward, most merged pull requests are set to "closed" status but a "Merged" label is assigned. However, during certain periods, few pull requests receive neither the "Merged" status nor the label despite being accepted. To address these false negatives, we adopt the approach proposed by Bertonecello et al. [8], which reports that some projects include a commit SHA in the close event of merged pull requests and we find it applies to Pytorch as well. Therefore, we combine three methods to identify merged pull requests in PyTorch: GitHub's "Merged" status, the "Merged" label assigned by maintainers, and pull requests closed with a commit SHA. A pull request identified by any of these methods is considered merged. Appendix A provides more details of our accepted PR identification approach and PyTorch's pull request approval history.

¹⁴<https://www.oreilly.com/content/how-the-tensorflow-team-handles-open-source-support/>

¹⁵<https://github.com/pytorch/pytorch/pull/8581>

¹⁶<https://github.com/apache/mxnet-ci/blob/master/services/github-bots/LabelBotFullFunctionality/README.md>

¹⁷<https://github.com/pytorch/pytorch/wiki/Bot-commands>

¹⁸<https://docs.github.com/en/rest/users>

2.3 Extracting Contributor Features

We extract 11,949 human contributors from the subject projects, where the cross-project contributors are treated as separate individuals in each project to account for potential variations in their behaviors across projects. For each contributor, we extract contributor features from their traces in the project repository and their GitHub profile page to capture their OSS activities and contributions. The extracted contributor features can be grouped into four categories: (i) personal attributes, capturing contributors' publicly available personal information defining their identity in the OSS community, such as their timezone and number of GitHub followers; (ii) commit, including measures of code change and non-code change efforts (e.g., documentation) of a contributor; (iii) issue, which includes activities and contributions related to issue reports, such as the number of issues raised or solved by a contributor; and lastly (iv) pull request, which involves activities and contributions related to pull requests, such as the number of pull requests submitted or reviewed by a contributor. In total, we extract 32 contributor features. A complete list of the contributor features is presented in Table 12 in Appendix B along with their definitions and rationales.

2.4 Correlation and Redundancy Analysis

Considering that correlated features can be expressed by each other and redundant features can be expressed by other features, highly correlated and redundant features blur the importance of the features to our analysis and result in unnecessary computations. Therefore, we conduct a correlation and redundancy analysis to remove the highly-correlated and redundant features.

- **Correlation Analysis:** We find that contributor features do not follow a normal distribution, thus we use Spearman Rank Correlation to conduct the correlation analysis [104]. Two features with a correlation coefficient higher than 0.7 are considered highly correlated [76]. For each pair of highly correlated features, we keep only one in the candidate list and remove the other one. The result of correlation analysis is shown in Figure 2. Ten features (i.e., Code Change Commits, Code Contribution, Non-Code Change Commit Rate, Average Issue Comments Received, Issue Solving Ratio, Issue Solving Rate, Issue Contribution, PR Merged, PR Contribution, and PR Contribution Rate) are removed and 22 features remain.
- **Redundancy Analysis:** R-squared is a measure that indicates how much variance of a variable can be explained by other variables. We use a R-squared cut-off at 0.9 to identify the redundant features [61] and find that there is no redundant feature in our data. As a result, 22 uncorrelated and non-redundant features remain.

3 Contributor Profiles

We aim to study the characteristics of contributors' activities within open-source ML projects. To achieve this, we construct contributor profiles to categorize and capture common characteristics among groups of contributors with similar behaviors. We select the contributor features that capture the key aspects of OSS contributor behaviors and use clustering algorithms on the selected features to identify groups of contributors with similar behaviors and construct their profiles accordingly. The detailed approach involves the following four steps:

Step 1: Select contributor features for clustering. To group contributors with similar behaviors, we select four contributor features that capture four key aspects of ML OSS contributors' behaviors to conduct the clustering. Inspired by the work of [77], we select contribute time, number of commits, and code contribution density to capture contributors' fundamental behaviors. Although their study focuses on software refactoring within Apache projects, these three features are broadly applicable for capturing the key aspects of OSS contributors' behaviors regardless of

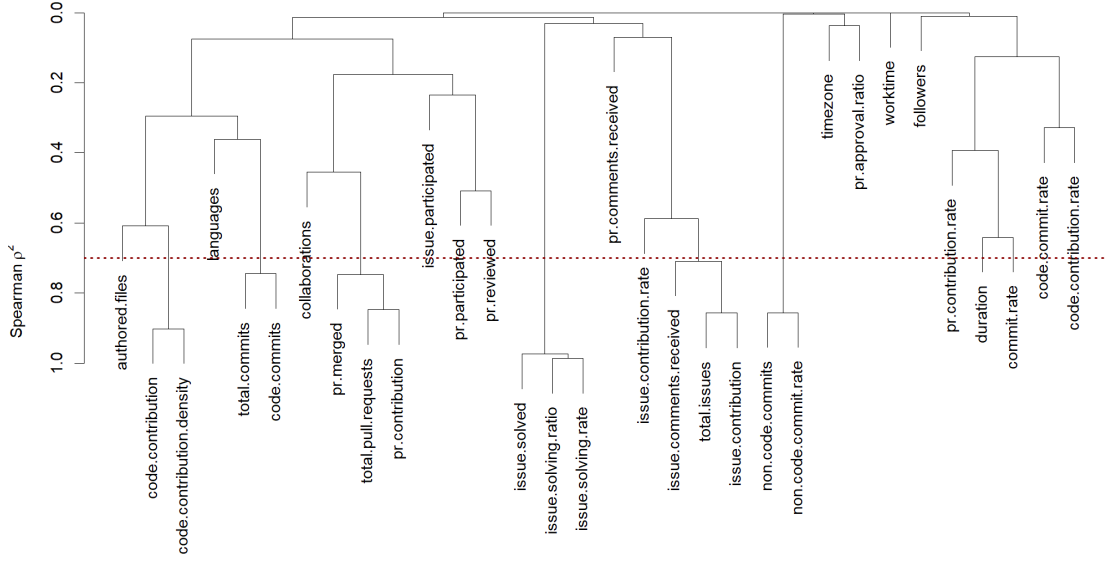


Fig. 2. Spearman correlation analysis of contributor features, presented in a hierarchical structure. Correlated feature pairs are below the red line.

the project domain. In addition, we include the number of programming languages to capture the technical expertise of ML contributors. The four selected features and the rationale behind them are as follows:

- **Contribute Time** to capture the contributor’s working habit (i.e., whether one would like to contribute to OSS during normal office hour in a work-like manner);
- **Number of Commits** to capture contributor’s contribution volume;
- **Code Contribution Density** to capture one’s contributing style (i.e., whether one would like to make small changes each time or make big changes, also indicating the complexity of one’s contribution);
- **Number of Programming Languages** to capture the contributor’s technical expertise in ML framework projects. Proficiency in multiple programming languages indicates expertise across various components of ML development. For instance, the implementation of tensors, ML operations, and computational graph execution usually utilizes C++, whereas high-level APIs are commonly written in Python.

Step 2: Normalize the selected contributor features. Clustering algorithms often use distance metrics to group similar data points. When features have different scales, those with larger scales can dominate the clustering process. Therefore, we normalize each selected contributor feature to ensure that all features contribute equally to the clustering. We apply Min-Max normalization to scale each selected feature across all contributors to a range of 0 to 1, while preserving the relative feature relationships between contributors, as shown in Equation 1.

$$\hat{x}_i = \frac{x_i - \min(x)}{\max(x) - \min(x)} \quad (1)$$

where $\hat{x}_i$ represents the normalized value of the i th contributor for a feature, x_i represents the original value of the i th contributor for the feature, $\min(x)$ and $\max(x)$ represent the minimum and maximum value of the feature across all contributors, $i \in \{1, 2, 3, \dots, 11, 949\}$, and a feature $\in \{\text{Contribute Time, Number of Commits, Code Contribution Density, Number of Programming Languages}\}$.

Step 3: Select the clustering algorithm. We examine 8 clustering algorithms in the Python scikit-learn package [81], namely, Kmeans, Affinity Propagation, Mean Shift, Spectral Clustering, Hierarchical Clustering, DBSCAN, OPTICS, and BIRCH, to cluster ML contributors with the four normalized features. The effectiveness of these clustering algorithms is evaluated using the Silhouette score, which is a metric to assess the quality of data point groupings [81]. Silhouette scores range from -1 to 1, where a score of 1 indicates an optimal clustering and -1 indicates the worst. Scores close to 0 indicate overlapping clusters and negative values indicate that a data point has been incorrectly assigned to a cluster since it is more similar to a different cluster.

For the clustering algorithms requiring a predefined number of clusters (i.e., Kmeans, Spectral Clustering, Hierarchical Clustering, and BIRCH), we experiment with the number of clusters range from 1 to 10 and select the optimal number of clusters with the highest Silhouette score. For the rest of the clustering algorithms, we conduct a gradient search to determine the optimal parameter set with the highest Silhouette score.

Table 2. Eight clustering algorithms and the associated evaluation results.

Clustering Algorithms	Kmeans	Affinity Propagation	Mean Shift	Spectral Clustering	Hierarchical Clustering	DBSCAN	OPTICS	BIRCH
Number of Clusters	3	5,542	4	4	3	12	2	4
Silhouette Score	0.559	0.053	0.564	0.565	0.535	0.522	0.527	0.542

Step 4: Identify contributor profiles. Based on the clustering algorithm evaluation results shown in Table 2, the clusters produced by Spectral Clustering have the highest Silhouette Score of 0.565, indicating the optimal clustering result among all clustering algorithms. Therefore, we identify our contributor profiles from the four clusters generated by Spectral Clustering. Figure 3 shows a summary of the profiles in terms of each clustering feature and we name each profile accordingly. As shown in Figure 3b, 3c, and 3d, the two clusters represented in green and orange have higher total commits, code contribution density, and number of used programming languages compared to the other two clusters, so we name them *Core* and *Peripheral* contributors respectively. *Core* and *Peripheral* contributors can be further divided into *Daytime* and *Nighttime* subgroups based on the time they used to make OSS contributions as shown in 3a. We summarize the four profiles in the following:

- **Core-Nighttime profile** contains 753 (6.3%) contributors. This profile has a larger number of commits, code contribution density, and programming languages used in comparison to *peripheral* contributors, with a median of 9 commits, 260 LOC per commit, and 3 programming languages. Contributors in this profile tend to make contributions during night times that outside normal working hours (i.e., outside 8 AM to 6 PM of their local time).
- **Core-Daytime profile** contains 2,016 (16.9%) contributors. This profile is similar to *Core-Nighttime* in terms of the number of commits, code contribution density, and programming languages, with a median of 19 commits,

207 LOC, and 3 programming languages. Contributors in this profile make contributions during day times within normal working hours.

- **Peripheral-Nighttime profile** contains 3,860 (32.3%) contributors, characterized by few commits, code contribution density, and programming languages, with a median of 1 commit, 9 LOC, and 1 programming language. Contributors within this profile make contributions outside normal working hours.
- **Peripheral-Daytime profile** contains 5,320 (44.5%) contributors. They are similar to *Peripheral-Nighttime* contributors in terms of few commits, code contribution density, and programming languages, with a median of 1 commit, 9 LOC, and 1 programming language. However, contributors in this profile tend to contribute during normal working hours.

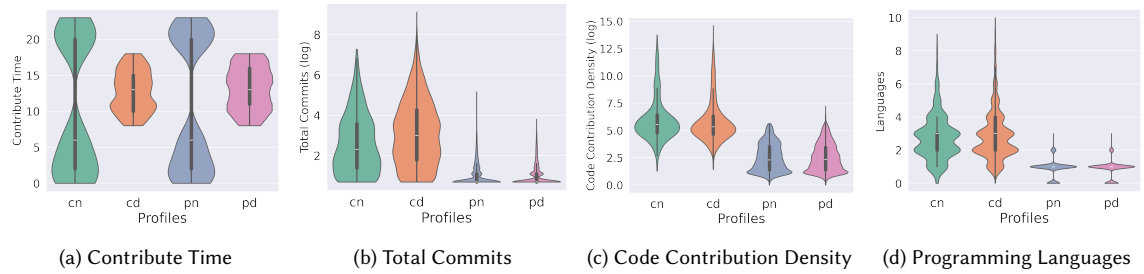


Fig. 3. Summary of four contributor profiles ('cn' refers to *Core-Nighttime*, 'cd' refers to *Core-Daytime*, 'pd' refers to *Peripheral-Daytime*, and 'pn' refers to *Peripheral-Nighttime*).

4 Contributor OSS Engagement

To gain a further understanding of contributors' engagement in various OSS activities, we study their OSS engagement from four aspects: workload composition, work preferences, technical importance, and ML-specific contributions.

4.1 Identifying Workload Composition Patterns

Workload composition reflects how contributors allocate their effort across key OSS routines, including committing, raising issue reports, participating in issue discussions, participating in pull request discussions, and code reviews within a specific period. We identify workload composition patterns to capture the common workload compositions and study the distribution of contributors with similar workload compositions as well as their impact on the subject project. Our approach to identifying contributor workload composition patterns involves the following four steps:

Step 1: Determine the length of the period to analyze contributors' workload composition. A contributor can be assigned to different tasks at different time periods. Therefore, contributors' workload composition is often dynamic and can evolve over time. We aim to capture their temporary workload compositions that are the focus of a contributor within a time interval, assuming the workload composition remains stable over brief periods.

We conduct a sensitivity analysis on 30, 60, 90, 120, and 180 days to identify the optimal period length and verify the stability of a contributor's workload composition. We observe that shorter periods, such as 30 and 60 days, lack precision in representing contributors' workload compositions, due to the potential of little activities within short periods. For example, suppose that a contributor only makes one commit in 30 days but engages in pull request discussions in the following month (i.e., 30 days), we may observe an exclusive focus on committing for the first period and an exclusive focus on pull request discussions for the subsequent period. In contrast, long time intervals, such as 120 and 180 days,

yield too few data points for our analysis, particularly in RQ3, where we model the relationship between contributor workload compositions and project popularity (e.g., star rating and number of forks). Specifically, when we divide the lifespan of a project into consecutive 90-day periods (e.g., TensorFlow’s first period spans from 2015/11/07 to 2016/02/05 and the second from 2016/02/05 to 2016/05/05), we have a total of 264 periods from all 8 subject projects. For each period, we count the increase in stars and forks, which serve as dependent variables for modeling project popularity. For each period, we also build workload composition vectors for all active contributors. The independent variables are the aggregated workload compositions of the active contributors in each period. This is because the project popularity is influenced not by individual behavior alone but by the collective behavior of all contributors during a given period. With this approach, we model the relationship between contributor activity and the increase of project popularity using a consistent time unit. Therefore, the number of periods determines the number of data points for building the model. Long period length would reduce the number of periods and result in insufficient data points for the model to converge.

Therefore, for each subject project, we divide the project lifespan into consecutive 90-day periods from the beginning of the project’s lifetime to the date of our data collection. We capture the workload composition of active contributors in each period. In the rest of this paper, the term ‘*period*’ refers to these 90-day project intervals.

Step 2: Construct contributor workload composition vector space. We create a 5-dimensional vector space to represent the contributors’ workload composition within a period. The five dimensions include the count of commits, raised issue reports, issue comments, pull request comments, and code reviews, which represent the key components of OSS project routines. For each period of a project, we create a workload composition vector for each active contributor. As the absolute counts of OSS activities may not accurately reflect the actual focus of contributions (e.g., conducting 2 code reviews might be a considerable contribution while making 2 issue comments might be a small contribution), we normalize each dimension of the workload composition vector across contributors. The normalization ensures that each dimension in the vector represents the relative contribution to an OSS activity among contributors and the vector can reflect the actual workload allocations of contributors. Furthermore, the relative contribution can vary between projects and even within the same project over different periods. Hence, for each period in each project, we apply Min-Max normalization to normalize each dimension of the workload composition vectors across all active contributors. The normalized vector space ensures unbiased comparisons between vector dimensions and vectors from different projects or periods. As a result, we build 41,628 workload composition vectors from 11,949 contributors across a total of 264 project periods from our 8 subject projects.

Step 3: Identify workload composition patterns. We employ agglomerative hierarchical clustering [81] to identify the workload composition patterns. Agglomerative hierarchical clustering begins by treating each vector as an individual cluster and successively merges smaller clusters with the highest similarity until all data points belong to a single cluster or a predefined dissimilarity threshold is reached. This iterative process naturally forms cohesive clusters and can produce a dendrogram, which is a hierarchical tree-like structure of data points and can be cut at different heights (i.e., dissimilarity threshold) to obtain different numbers of clusters. Compared to other clustering algorithms discussed in Section 3, we select hierarchical clustering to identify workload composition patterns because the dendrogram enables us to visualize and explore the similarity of workload compositions at different levels of granularity, thereby determining the level of similarity of the resulting patterns.

We use cosine similarity as the similarity measurement for workload composition vectors, as it gauges the similarity of vector dimensions regardless of the vector scale. This aligns with our goal of identifying contributors with a similar focus across different types of OSS contributions, regardless of their amount of contribution.

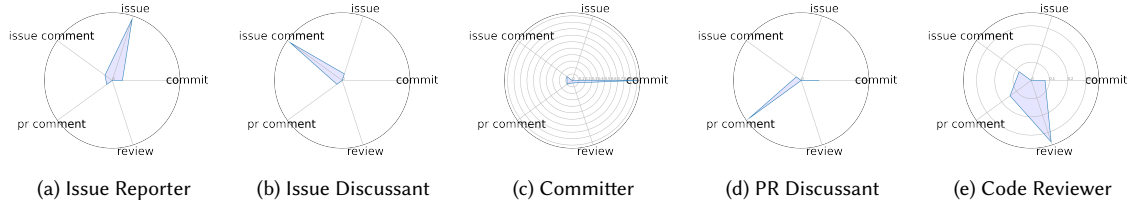


Fig. 4. Workload composition vector of the centroid of each workload composition pattern.

We first compute the pairwise cosine similarity of every two workload composition vectors to build a distance matrix. Then, we apply agglomerative hierarchical clustering on the distance matrix to group similar vectors and plot the dendrogram of the clustering. We manually inspect the clusters generated from different cut-off dissimilarity thresholds and calculate the Silhouette scores for the resulting clusters to determine the optimal number of clusters and identify workload composition patterns accordingly.

Step 4: Summarize workload composition patterns. We find five clusters as the optimal number of clusters at a cut-off threshold of 0.9, which also achieves the highest Silhouette score of 0.562. From these clusters, we identify five workload composition patterns. We observe that the 20-80 rule [64, 65] applies to contributors across all patterns, where a small subset of contributors are more active on more types of activity, while the majority do few works. The identified workload compositions are still relevant, as we wish to study both types of contributors and understand the specific combinations of activities they tend to focus on.

Figure 4 shows the centroid workload composition vector in each cluster, which is the point with the highest cosine similarity with other points in the cluster. We summarize the work focus of each cluster according to the centroid behavior in the following:

- **Pattern 1: Issue Reporter** accounts for 19.8% of occurrences. Contributors in this pattern focus on raising issue reports and participate minimally in other activities.
- **Pattern 2: Issue Discussant** comprises 16.1% of occurrences. They focus on discussing issue reports while infrequently engaging in other types of contributions.
- **Pattern 3: Committer** is the most common pattern and comprise 40.9% of occurrences. They only focus on making commits and solely participate in the other four activities.
- **Pattern 4: PR Discussant** comprise 15.5% of occurrences. They mainly focus on participating in pull request discussions, with occasional involvement in issue discussions and making code commits.
- **Pattern 5: Code Reviewer** is the least common pattern comprising 7.7% of occurrences. Contributors in this pattern primarily focus on code reviews but occasionally make commits and participate in collaborative activities (i.e., pull request and issue discussions).

4.2 Extracting Work Preference Features

While workload composition provides a static view of contributors' focus of OSS activities within a period of time, work preference features capture their contribution dynamics and preferences in that period, such as whether they make consistent contributions (e.g., making one commit daily) versus peak contributions (e.g., making a high number of commits on a single day and remaining inactive afterwards), as well as their preference on making specific types of contributions versus an even contribution across various types.

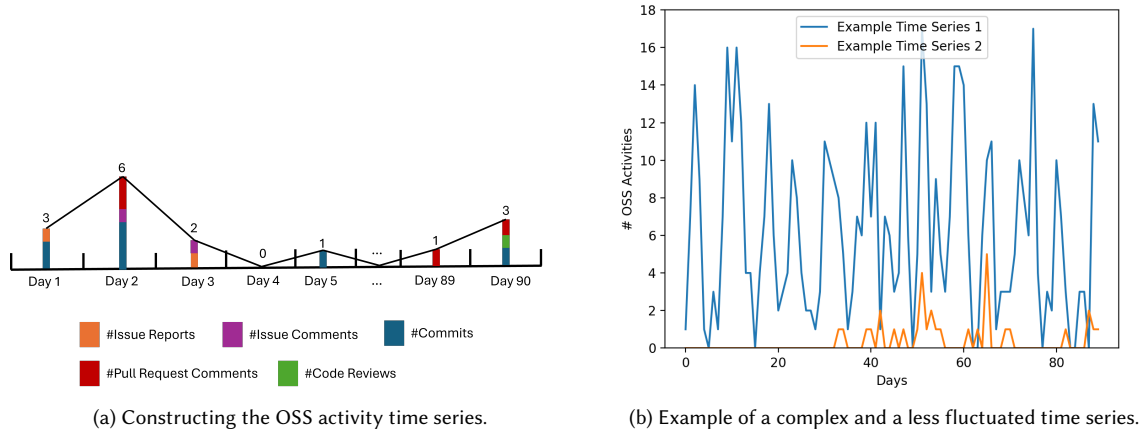


Fig. 5. An example of contributors' OSS activity time series within a 90-day period.

As shown in Figure 5a, we count a contributor's daily activity in terms of the five OSS activity dimensions (i.e., the number of issue reports, issue comments, commits, pull request comments, and code reviews) each day and create a time series for each contributor over a 90-day period (as discussed in 4.1). We use the Python *tsfresh* package [15] to extract time series features including *binned_entropy*, *c3*, *number_cwt_peaks*, *longest_strike_above_mean*, and *longest_strike_below_mean*. These time series features measure a contributor's contribution dynamics. Figure 5b presents examples of contributors' OSS activities time series with different contribution dynamics, where example time series 1 exhibits complex dynamics and example time series 2 exhibits comparatively fewer fluctuations. We also extract two features, *diverse* and *balance*, to measure the breadth of OSS contribution types made by a contributor and the degree to which they distribute their efforts evenly across various types within each period. We consider these 9 extracted features as contributor work preference features. Detailed descriptions and implications of the work preference features are outlined in Table 3.

Table 3. Extracted features of contributor work preferences.

Contributor Work Preference Features	Description
<i>binned_entropy</i> (10)	Binned entropy measures the complexity of the sequence. A higher binned entropy indicates developers have more complex contribution dynamics and make less constant contributions.
<i>c3</i> (1), <i>c3</i> (2), <i>c3</i> (3)	C3 statistics measures the degree of nonlinearity or complexity in a time series by calculating the correlation between the original time series and its lag of 1, 2, 3 respectively. Higher C3 statistics indicate developers have more complex contribution dynamics and make less constant contributions.
<i>number_cwt_peaks</i>	Number of peaks of contributions. A higher value indicates developers make more peak contributions.
<i>longest_strike_above_mean</i>	The ratio of the longest subsequences above mean. A higher value indicates more consecutive days of high contribution.
<i>longest_strike_below_mean</i>	The ratio of the longest subsequences below mean. A higher value indicates more consecutive days of low contribution.
<i>diverse</i>	The number of different types of contributions a developer has made.
<i>balance</i>	The inverse of the variance of a developer's normalized contribution on the five OSS activities. A higher value indicates developers make a more even amount of different types of contributions.

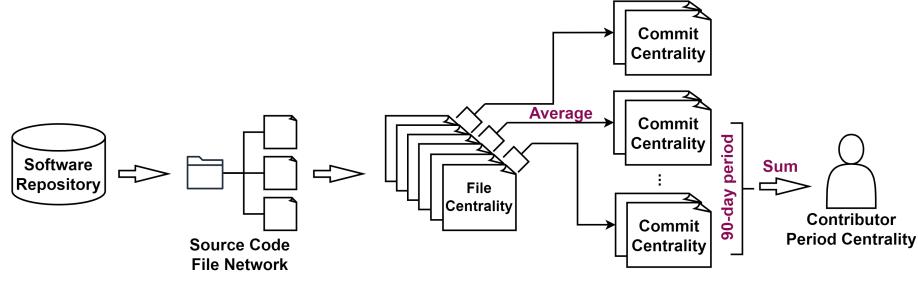


Fig. 6. Measure contributor technical importance.

4.3 Evaluating Contributor Technical Importance

The technical importance of a contributor can be measured using the importance of the commits made by the contributor. The importance of a commit can be measured with the eigenvector centrality of source code files modified by the commit. Eigenvector centrality measures the importance of a node within a network. In our case, the software repository can be viewed as a network, with the folders and source code files as nodes, and their containment relationships as edges. Therefore, the eigenvector centrality of a source code file measures the importance of the file within the file network of a repository. Contributors who frequently make changes to important files can be considered as possessing higher technical importance to the project.

Figure 6 shows our approach for measuring contributor technical importance. Following a similar methodology as proposed by Yue et al. [103], we first construct a network with folders and source files as nodes and the relationship of a folder containing a file as edges, then calculate the eigenvector centrality of each source file, as *file centrality*. The *commit centrality* is calculated by averaging the eigenvector centrality of the source files that have been changed in the commit. Higher *commit centrality* means that the commit makes changes to more influential files in the repository and can have a greater impact on the codebase. We sum the *commit centrality* of all commits made by a contributor within a period to calculate the *period centrality*, which measures the technical importance of a contributor within a period, considering both the intensity and importance of their commits. We introduce four metrics to quantify the technical importance of a contributor:

- ***max_commit_centrality***: The highest commit centrality among all commits made by a contributor.
- ***max_centrality_day***: The number of days taken by a contributor to make the commit with the highest centrality.
- ***max_period_centrality***: The highest period centrality among all active periods of a contributor.
- ***max_centrality_period***: The number of periods taken by a contributor to achieve the highest period centrality.

4.4 Qualitative Analysis for ML-Specific Contribution

To further understand the nature of ML OSS contributors' code contributions beyond the volume of code changes, we manually analyze the content of their pull requests, focusing specifically on the ML-specific aspects they contribute to. Pull requests are the primary approach through which contributors modify the project's codebase, which provides valuable context through descriptions, comments, and code review activities. This makes pull requests more informative than commits for understanding the nature of contributions. Therefore, we focus on analyzing pull requests to determine how contributors from different profiles contribute to different ML-specific aspects of the codebase. For this analysis,

we adopt the taxonomy proposed by Chen et al. [13], which categorizes the architecture of a DL framework into five levels, from the highest to the lowest:

- (1) **User-Level API:** is high-level APIs that are designed to simplify the use of the framework for end users, including APIs for input data processing, model building, model deployment, and other user-facing utilities that facilitate the DL workflow.
- (2) **Graph-Level Implementation:** includes processes and functions that manage the organization and the execution of operations within a computational graph. A computational graph is a directed graph where nodes represent operations (e.g., matrix multiplication, convolution) and edges represent the flow of data (typically in the form of tensors) between these operations. This level includes graph construction, transformation, and execution.
- (3) **Operation Implementation:** provides detailed implementations of operations (e.g., convolution, pooling, mathematical functions) that form the nodes in computational graphs and primarily operate at the tensor level.
- (4) **General Utility:** are reusable functions and data structures (e.g., type conversion, padding) that support the higher-level architecture of the DL framework.
- (5) **Environment-Dependent Processing:** supports DL frameworks run across different hardware (e.g., GPU) and software environments by managing environment-specific processes (e.g., memory allocation).

We select a stratified random sample of 384 pull requests from a total of 130,779 pull requests across all subject projects, with a 95% statistical confidence level and a 5% margin of error for the sample size [77, 78]. Stratification is based on the four contributor profiles, with 96 pull requests randomly sampled from the pool of pull requests raised by contributors in each profile. Two independent labelers (i.e., the first two authors) manually review the title, description, discussions, and code changes in an initial set of 40 sampled pull requests, categorizing it as a contribution to one of the five ML components or as a general software development contribution unrelated to the ML-specific architecture. After reaching a consensus on labels based on the initial set of samples, the two labelers continue to label the rest of samples. To assess inter-rater reliability, we calculate Cohen’s Kappa coefficient [19], a standard metric for evaluating agreement between raters, and achieve an overall Kappa value of 0.88. Discrepancies between labelers are resolved through discussion, where both labelers compare interpretations and reach consensus on the final label of each pull request.

5 Experiment Results

5.1 RQ1: What are the characteristics of each contributor profile?

5.1.1 Motivation

We identify 4 contributor profiles based on multiple factors including contribute time, the number of commits, code contribution density, and the number of used programming languages. In this research question, we further delve into the important features of each profile and explore the differences between the profiles as well as their shared attributes. Understanding contributor profiles can equip project managers and maintainers with the knowledge to engage the contributors of different profiles in the project effectively and support strategies to harness the varied spectrum of contributors.

5.1.2 Approach

To gain a further understanding of contributor profiles, we build four binary logistic regression models to identify the significant contributor features associated with each of the four contributor profiles. Our detailed approach includes the following three aspects:

Model Construction: To build the model for each contributor profile, we exclude the four features used for identifying the profiles (i.e., Contribute Time, Number of Commits, Code Contribution Density, and Number of Programming Languages) and use the remaining 18 non-correlated and non-redundant contributor features (as described in Section 2.4) as independent variables. We establish a binary dependent variable by labeling contributors of the target profile (e.g., *Core-Nighttime*) as 1 and contributors of the remaining three profiles (i.e., *Core-Daytime*, *Peripheral-Nighttime*, and *Core-Daytime*) as 0. Therefore, we create four models. Binary dependent variable allows us to explore the differences between the target profile and the rest, as well as to identify the important features defining the target profile. To avoid bias caused by the imbalanced labels, we construct a balanced dataset by using an equal number of data points for both 1 and 0 labels. We include all data points labeled 1 and a corresponding number of randomly selected data points labeled 0. When constructing the logistic regression model, we build the base model using *glm* function in R [29], and subsequently employ the *step* function to build step models, which iteratively eliminate the features that are not significant to the model.

Model Performance Evaluation: We evaluate our model’s effectiveness using the Area Under the Curve (AUC) and Matthews Correlation Coefficient (MCC) metric. The AUC, representing the space beneath the Receiver Operating Characteristic (ROC) curve, quantifies the capability of a binary classifier to differentiate between classes. AUC values range from 0 to 1, with higher values signifying better performance. Specifically, an AUC of 0.5 indicates a random classifier, while an AUC of 1 denotes a perfect classifier [44]. MCC measures the correlation between the binary predictions and true labels, considering all four categories in the confusion matrix (i.e., true positives, false negatives, true negatives, and false positives), which is a more reliable metric than the F1 score and accuracy, especially on imbalanced datasets [14]. MCC values range from -1 to 1, where 0 indicates random guessing, 1 represents a perfect classifier, and -1 represents complete misclassification.

To further evaluate the robustness of our models, we apply the bootstrapping evaluation technique, which helps mitigate the potential bias introduced by imbalanced data or by data points not included in the training set. For each model, we randomly sample 1,000 data points from both the 0 and 1 labels (500 data points for the *Core-Nighttime* model due to the smaller data size of the 753 contributors in this profile). The model is then used to predict the labels of the combined 2,000 data points (1,000 for the *Core-Nighttime* model). This process is repeated 1,000 times for each model, and we compute the mean accuracy, AUC, and MCC to evaluate their overall performance.

Model Result Analysis: To identify the significant features of each model and its corresponding profile, we employ Wald statistics ($\tilde{\chi}^2$) to estimate the feature importance and statistical significance (p-values) of each contributor feature, as Wald statistics are often used in regression analysis to assess the significance of individual coefficients or parameters in a regression model [9]. A higher $\tilde{\chi}^2$ value indicates a larger impact of the particular feature on the discriminatory capability of the model [68, 93]. We utilize the *ANOVA* function in R to compute $\tilde{\chi}^2$ values and the corresponding p-values. We further calculate the percentage of $\tilde{\chi}^2$ of each feature relative to the sum of $\tilde{\chi}^2$ values to show the proportion to the overall importance for each feature [41]. The significant features for each profile are marked with asterisks in Table 5, with more asterisks indicating higher statistical significance levels.

We also analyze the sign of coefficients of each feature in the logistic regression models. Positive coefficients imply a direct correlation with the dependent variable, suggesting that as the feature increases, the chances of a contributor being classified into the profile labeled as 1 also tend to increase. In contrast, negative coefficients imply an inverse relationship. The upward ($\nearrow$) and downward ($\searrow$) arrows in Table 5 denote positive and negative correlations.

To validate the model performance, we apply two-sided Mann-Whitney U test [72] to compare contributor features across contributors in *Core* and *Peripheral* profiles with the null hypothesis $H0_1$, as well as compare their *Daytime* and

Nighttime subgroups with the null hypothesis H_{02} and H_{03} respectively. We apply Cliff's delta [17] to estimate the effect sizes.

- H_{01} : The distributions of the given contributor feature are the same across *Core* contributors and *Peripheral* contributors.
- H_{02} : The distributions of the given contributor feature are the same across *Core-Nighttime* and *Core-Daytime* contributors.
- H_{03} : The distributions of the given contributor feature are the same across *Peripheral-Nighttime* and *Peripheral-Daytime* contributors.

5.1.3 Results

Table 4. AUC of 4 binary logistic regression models.

Evaluation Result		Model 1 Core- Nighttime	Model 2 Core- Daytime	Model 3 Peripheral- Nighttime	Model 4 Peripheral- Daytime
Model Evaluation	Accuracy	0.72	0.77	0.64	0.63
	AUC	0.72	0.77	0.64	0.63
	MCC	0.45	0.53	0.28	0.26
Bootstrapping Evaluation	Accuracy	0.71	0.75	0.63	0.62
	AUC	0.71	0.75	0.63	0.62
	MCC	0.42	0.51	0.27	0.25

The constructed logistic regression models have the highest discrimination performance for classifying *Core-Daytime* contributors. Table 4 presents the AUC values of the models. The models for *Core-Nighttime* and *Core-Daytime* contributors have the top AUC values of 0.72 and 0.77 respectively. Both *Peripheral-Nighttime* and *Peripheral-Daytime* profile models achieve AUC values over 0.6, indicating the effectiveness of the models in classifying the respective profiles beyond random classification. Additionally, all models yield MCC scores greater than 0, which stands for a positive correlation between the predicted and actual labels. Bootstrapping evaluations show a minimal decrease in performance (i.e., 0.01 to 0.02 for AUC and 0.01 to 0.05 for MCC), which shows the robustness of the models across our entire dataset. Additionally, statistical test results show that the null hypothesis H_{01} is rejected for 31 (out of 32) contributor features, which shows significant differences between *Core* and *Peripheral* contributors regarding the majority of contributor features. Similarly, the null hypothesis H_{02} is rejected for 21 contributor features and H_{03} is rejected for 18 contributor features, highlighting the significant differences between *Core-Nighttime* and *Core-Daytime* contributors and between *Peripheral-Nighttime* and *Peripheral-Daytime* contributors respectively. The complete list of test results is presented in Table 13 in Appendix C. In the rest of the section, we describe our analysis of the logistic regression models and statistical test results, and identify the important features associated with each profile.

The common important features across all four contributor profiles include duration, authored files, average number of comments received on pull requests, pull requests approval ratio, and collaboration, which serve as key factors in differentiating *Core* and *Peripheral* contributors. As shown in Table 5, both *Core* profiles are associated with longer project duration, a higher number of authored files, more comments received on their pull requests, and more collaborated contributors, while *Peripheral* profiles are associated with a higher pull request

Table 5. Result of 4 binary logistic regression models.

Contributor Features	Core-Nighttime			Core-Daytime			Peripheral-Nighttime			Peripheral-Daytime		
	Chisq%	Signf.	Rel.	Chisq%	Signf.	Rel.	Chisq%	Signf.	Rel.	Chisq%	Signf.	Rel.
Duration	2.94	**	↗	17.93	***	↗	2.5	***	↘	11.14	***	↘
Timezone	6.91	***	↗	25.01	***	↘	50.05	***	↗	13.78	***	↘
Authored Files	17.01	***	↗	12.36	***	↗	9.52	***	↘	27.63	***	↘
Commit Rate				1.58	***	↘		*	↘	1.99	***	↗
Code Commit Rate				1.82	***	↗	1	*	↘			
Code Contribution Rate	0.94	.	↗	0.3		↗	5.38	***	↘	0.45	.	↘
Non Code Commits	0.65		↘									
Total Issues							0.61	.	↗			
Issue Solved	1.95	*	↗	1.49	***	↘				1.71	***	↗
Issue Participated	2.03	*	↘									
Issue Contribution Rate	1.32	*	↘	0.57	*	↘	1.65	**	↗	0.39		↘
Total Pull Requests				4.39	***	↗	2.76	***	↘	3.21	***	↘
PR Reviewed	8.03	***	↘	2.63	***	↘	2.71	***	↗	5.5	***	↗
PR Approval Ratio	12.05	***	↘	14.33	***	↘	14.89	***	↗	1.94	***	↗
Avg PR Comments Received	18.87	***	↗	10.61	***	↗	0.95	*	↘	14.36	***	↘
PR Participated				0.48	*	↘				0.73	*	↗
PR Contribution Rate	7.04	***	↘	2.05	***	↘	0.54	.	↗			
Followers				0.65	*	↘						
Collaborations	15.37	***	↗	3.79	***	↗	3.69	***	↘	8.97	***	↘
Project Encoded	4.88	***	↘				3.76	***	↘	8.2	***	↗

Signif. codes: 0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Chisq%: The importance of a feature in modeling a profile in percentage.

Rel.: ↗ stands for a positive correlation between a feature and the likelihood of being a profile, and ↘ stands for a negative correlation.

approval ratio compared to *Core* contributors (as *Core* contributors may be involved in more complex code change). The Mann-Whitney U test further confirms that these five features show statistically significant differences between *Core* and *Peripheral* contributors. Furthermore, the difference in collaborations also indicates that *Core* contributors tend to have active involvement in mentoring activity or project management, while *Peripheral* contributors tend to focus on their specific contributions to the project without involving others.

Timezone is the main factor to differentiate *Daytime* and *Nighttime* contributors. Table 5 shows that timezone is significant for all profiles with positive associations with two *Nighttime* profiles (i.e., *Core-Nighttime* and *Peripheral-Nighttime*) and negative associations with two *Daytime* profiles (i.e., *Core-Daytime* and *Peripheral-Daytime*). However, the significance of time zones lies in their representation of geographical regions. We categorize contributor timezones into three regions: Americas (timezone from -12 to -2, containing 48.8% of contributors), Europe/Africa (-1 to 3, containing 32.5% contributors), and Asia (4 to 12, containing 18.7% contributors). We find that 72.3% of contributors from the Americas are *Daytime* contributors, contrasting with 45.7% from Asia and 54% from Europe/Africa. This suggests a higher inclination among Americas-located contributors towards adopting a regular job-like approach to ML OSS projects compared to other regions. *Nighttime* contributors might have flexible contribute schedules or prioritize their contributions to the project during their free time or outside their regular job commitments. It is also possible that *Nighttime* contributors from Europe/Africa or Asia attempt to align their contributions with work hours in the Americas to ensure quick responses from the majority of the development team. These findings indicate the predominant influence of contributors from the Americas in driving the selected ML projects.

***Core-Daytime* contributors tend to stay longer in the project and are more involved in commits and collaborative activities, while *Core-Nighttime* contributors tend to handle more complex tasks.** Logistic regression results indicate that the two *Core* profiles exhibit similar performance across most features except for

timezone. The Mann-Whitney U test reveals significant differences between two *Core* profiles on 21 features. The null hypothesis H_{02} is rejected for the duration, with a negative effect size, indicating that *Core-Daytime* contributors tend to stay involved in projects longer than *Core-Nighttime* contributors. H_{02} is also rejected for the number of authored files and 7 commit-related features (i.e., the total number of commits, commit rate, number of code commits, code commit rate, number of non-code commits, non-code commit rate, and code contribution) with negative effect sizes. This suggests that *Core-Daytimes* tend to have a boarder scope of authorship and make more frequent and extensive coding contributions compared to *Core-Nighttime* contributors. Additionally, H_{02} is rejected for the number of issues participated in, pull requests participated in, and pull requests reviewed with negative effect sizes. This suggests that *Core-Daytime* contributors are more actively involved in discussions in other contributors' pull requests and reviews, guiding users in resolving issues and offering feedback on pull requests. These activities indicate a strong focus on mentoring, as they involve knowledge sharing and helping other contributors align their contributions with project requirements. Conversely, H_{02} is rejected with positive effect sizes for code contribution density and the average number of comments received on issues and pull requests, indicating that *Core-Nighttime* contributors are more likely to report complex issues or submit pull requests with complex or influential changes, often involving larger code churns in their commits than *Core-Nighttime* contributors. Additionally, H_{02} is also rejected with a positive effect size for the pull request approval ratio, suggesting that despite the complexity of their contributions, *Core-Nighttime* contributors maintain a high standard of quality.

Despite sharing similar contribute time, commit volume, code contribution density, and proficiency in multiple programming languages, contributors within the same profile may have different preferences and areas of focus. Logistic regression model results show that the two *Core* profiles tend to be characterized by positive association with independent coding features indicating their high code contributions. Moreover, the two *peripheral* profiles tend to be characterized by positive association with issue handling (i.e., total issues, issues solved, and issue contribution rate) and collaborative coding (i.e., the number of pull requests reviewed, pull requests participated in, and pull request approval ratio), indicating active participation in collaborative activities. However, the Mann-Whitney U test results show that *Core* contributors are statistically higher in 4 of these features than *Peripheral* contributors excluding the pull request approval ratio. This contradicted observation indicates that the logistic regression models can effectively leverage features related to the code contributions of *Core* contributors but may overlook the characteristics of those *Core* contributors who actively participate in other OSS activities.

For instance, 55.4% *Core-Nighttime* and 50.5% *Core-Daytime* contributors have raised at least one issue report. We use the Mann-Whitney U test and Cliff's delta to compare the number of pull requests and pull request approval ratio between contributors who have raised at least one issue and those who have not, for both *Core* profiles respectively. We find that contributors who have raised issue reports make significantly more pull requests (with a large effect size for both profiles) and higher pull request approval ratio (with a small effect size for *Core-Daytime* contributors and negligible for *Core-Nighttime*) compared to those who have never raised any issues. Further investigation is needed to understand the distribution of workload within each profile and how contributors engage in different OSS activities.

RQ1 Summary

Project experience, breadth of file authorship, pull requests comments received and approval ratio, and collaborations are important features common to all four contributor profiles, primarily distinguishing *Core* from *Peripheral* profiles. Geographical location is an important factor differentiating contributors in *Daytime* and *Nighttime* profiles. *Core-Daytime* contributors tend to stay longer in the project and are more involved in commits and collaborative activities. *Core-Nighttime* contributors tend to handle more complex tasks. Even within the same profile, contributors exhibit varied engagements in OSS activities.

5.2 RQ2: What is the OSS engagement of each contributor profile?

5.2.1 Motivation

In RQ1, we observe that contributors within the same profile may differ in their involvement in different OSS activities. Given the broad range of open-source tasks and the limited time OSS contributors can allocate to such activities, they often prioritize certain tasks over others based on their needs and preferences. Understanding contributor workload compositions is essential to enhance our comprehension of contributor profiles, as it provides detailed insights into their individual preferences and priorities. Additionally, while workload composition provides a static view of contributors' activities in their overall engagement in the OSS activities within a period, the dynamics of their activities within that period remain unexplored. With the same amount of workload, contributors may choose to fulfill their workload gradually over a period or complete tasks in a single burst, depending on their preferences. Exploring work preferences is important as reported by Yue et al. [103] that contribution dynamics can affect the performance of contributors and their technical importance to a project. Moreover, committing is often considered the most important OSS workload. Although we have analyzed the volume of commits made by contributors across different profiles, the technical importance of their commits to the project remains unclear. Understanding contributor workload composition and work preferences can assist project managers in fostering collaborations among contributors with shared interests and similar work patterns. Moreover, understanding the technical importance and the ML-specific contribution can help project maintainers recognize important contribution and assign tasks that align with each contributor's expertise.

5.2.2 Approach

We explore and compare the behaviors of contributors across different profiles during their engagement in OSS activities from four aspects: their workload compositions, work preferences, technical importance, and ML-specific contributions. Our detailed approach is described as follows:

5.2.2.1 Workload Composition Analysis

As described in Section 4.1, we identify five workload composition patterns that reflect five types of workload compositions of contributors within a period. We investigate the frequency of each type of workload composition for each contributor during one's active periods. The most frequent workload composition pattern of a contributor is identified as one's major pattern. For each profile, we calculate the proportion of contributors with each workload composition pattern as their major pattern and compare the proportions across profiles.

Chi-square test [80] is a statistical test that can determine whether the proportions of categorical variables are the same across different groups or populations. We use the Chi-Square test to compare the distributions of major workload compositions (i.e., most frequent workload composition) of *Core* and *Peripheral* contributors with the null hypothesis

H_{04} (with a significance threshold of p-value < 0.05), as well as their respective *Daytime* and *Nighttime* subgroups with the null hypothesis H_{05} and H_{06} .

- H_{04} : The distributions of the major workload composition patterns are the same across Core contributors and Peripheral contributors.
- H_{05} : The distributions of the major workload composition patterns are the same across Core-Nighttime and Core-Daytime contributors.
- H_{06} : The distributions of the major workload composition patterns are the same across Peripheral-Nighttime and Peripheral-Daytime contributors.

Cramér's V is an effect size measure derived from the chi-squared statistic and is often used to quantify the magnitude of the difference identified by the Chi-square test. We calculate Cramér's V for the groups with significant differences identified by our Chi-square tests to measure effect sizes. We employ the same interpretation of Cramér's V as [20] at degrees of freedom of 4 (i.e., negligible: $0 < .05$, small: $.05 < .15$, medium $.15 < .25$, and large: $\geq .25$).

5.2.2.2 Work Preference Analysis

To measure the temporal contribution dynamic and preferences of contributors, we extract 9 work preference features from the OSS activity time series of each contributor within each 90-day period as described in Section 4.2. We apply two-sided Mann-Whitney U test [72] to compare each work preference feature between contributors in *Core* and *Peripheral* profiles with the null hypothesis H_{07} . We also compare their respective *Daytime* and *Nighttime* subgroups with the null hypothesis H_{08} and H_{09} , as specified below. We apply Cliff's delta [17] to estimate the effect sizes.

- H_{07} : The distributions of the given work preference feature are the same across Core contributors and Peripheral contributors.
- H_{08} : The distributions of the given work preference feature are the same across Core-Nighttime and Core-Daytime contributors.
- H_{09} : The distributions of the given work preference feature are the same across Peripheral-Nighttime and Peripheral-Daytime contributors.

5.2.2.3 Technical Importance Assessment

Four technical importance metrics are introduced in Section 4.3 to evaluate the importance of a contributor's code contributions. We apply the Mann-Whitney U test to compare each technical importance metric across *Core* and *Peripheral* contributors with the null hypothesis H_{010} , as well as their respective *Daytime* and *Nighttime* subgroups with the null hypothesis H_{011} and H_{012} respectively. We also calculate Cliff's delta to estimate the effect sizes.

- H_{010} : The distributions of the given technical importance metric are the same across Core contributors and Peripheral contributors.
- H_{011} : The distributions of the given technical importance metric are the same across Core-Nighttime and Core-Daytime contributors.
- H_{012} : The distributions of the given technical importance metric are the same across Peripheral-Nighttime and Peripheral-Daytime contributors.

5.2.2.4 ML-Specific Contribution Analysis

To categorize the content of contributors' code contributions, we label 96 randomly sampled pull requests from each profile (in total 384) into one of six categories: User-Level API, Graph-Level Implementation, Operation Implementation, General Utility, Environment-Dependent Processing, or General Software Development (as described in Section 4.4).

We apply the Chi-square test to compare the distribution of different types of ML-specific contributions between *Core* and *Peripheral* contributors with the null hypothesis H_{013} , as well as their respective *Daytime* and *Nighttime* subgroups with the null hypothesis H_{014} and H_{015} respectively. We calculate Cramér’s V to measure effect sizes. We employ the same interpretation of Cramér’s V as [20] at degrees of freedom of 5 (i.e., negligible: $0 < .04$, small: $.04 < .13$, medium $.13 < .22$, and large: $\geq .22$).

- H_{013} : The distributions of the given category of contribution are the same across *Core* contributors and *Peripheral* contributors.
- H_{014} : The distributions of the given category of contribution are the same across *Core-Nighttime* and *Core-Daytime* contributors.
- H_{015} : The distributions of the given category of contribution are the same across *Peripheral-Nighttime* and *Peripheral-Daytime* contributors.

5.2.3 Results

5.2.3.1 Workload Composition

Table 6. Distribution of major workload composition patterns across contributor profiles.

Major Workload Composition Pattern	Core-Nighttime	Core-Daytime	Peripheral-Nighttime	Peripheral-Daytime
Issue Reporter	14.6%	10.4%	17.6%	14.8%
Issue Discussant	8.2%	7.5%	9.1%	9.3%
Committer	45.9%	54.4%	57.6%	58.5%
PR Discussant	23.9%	17.8%	14.2%	15.5%
Core Reviewer	7.3%	9.9%	1.6%	1.8%

Based on our Chi-Squared test and Cramér’s V results, we reject the null hypothesis H_{04} , H_{05} , and H_{06} . The test results indicate significant differences in the distribution of major workload composition patterns between contributor profiles, with a medium effect size for the difference between *Core* and *Peripheral* contributors and a small effect size for both the difference between their *Daytime* and *Nighttime* subgroups.

Committer is the most common workload composition pattern for all contributor profiles. PR Discussant is the second most frequent for Core contributors and Peripheral-Daytime contributors, whereas Issue Reporter is the second most for Peripheral-Nighttime contributors. As shown in Table 6, contributors with Committer as their major pattern constitute the highest proportion in all profiles ranging from 45.9% to 58.5%. This finding challenges the intuitive assumption that *Peripheral* contributors would primarily focus on issue reporting or discussion and less focus on code contributions. Instead, the majority of *Peripheral* contributors focus on code contribution through committing. *Core-Nighttime* contributors have the highest percentage of PR Discussants (23.9%) and the lowest percentage of Committers (45.9%). This result aligns with the findings in RQ1 (i.e., *Core-Nighttime* contributors tend to make more complex contributions that require substantial discussions in pull requests), often with fewer but larger commits compared to *Core-Daytime* contributors. The high prevalence of PR Discussant in both *Core* profiles, combined with the result from RQ1 (i.e., *Core* contributors are significantly more involved in discussions on others’ pull requests), indicates that a considerable proportion of *Core* contributors are multitaskers who often

Table 7. Result of Mann-Whitney U test and effect size results for work preference features.

Work Preference Features	Group1: Core Group2: Peripheral	Group1: Core-Nighttime Group2: Core-Daytime	Group1: Peripheral-Nighttime Group2: Peripheral-Daytime
binmed_entropy	large (0.49)	negligible (-0.056)	negligible (0.051)
c3(1)	medium (0.331)	negligible (-0.035)	negligible (0.026)
c3(2)	small (0.279)	negligible (-0.037)	negligible (0.02)
c3(3)	small (0.292)	negligible (-0.042)	negligible (0.016)
number_cwt_peaks	large (0.495)	negligible (-0.079)	negligible (0.048)
longest_strike_above_mean	medium (0.372)	negligible (-0.026)	negligible (0.044)
longest_strike_below_mean	medium (-0.463)	negligible (0.088)	not significant
diverse	small (0.271)	not significant	negligible (0.067)
balance	small (-0.327)	negligible (0.077)	negligible (-0.018)

Groups with no significant difference (i.e., null hypothesis is accepted) are indicated with 'not significant'. Groups that are significantly different examined by Mann-Whitney U test are indicated with Cliff's delta value and its interpretation (i.e., negligible, small, medium, or large). A positive value means group1 is greater than group2, and vice versa. Three columns from left to right correspond to the null hypothesis H_{07} , H_{08} , and H_{09} respectively.

engage in a mix of coding, mentoring, and project management activities. Although Code Reviewers make up the smallest proportion of *Core* contributors, the majority of Code Reviewers are *Core* contributors, indicating their focus on maintaining and enhancing the quality of the codebase.

For *Peripheral* contributors, both Issue Reporter and PR Discussant are secondary workload composition patterns with similar prevalence (ranging from 14.2% to 17.6%). Given that less than 24% of *Peripheral* contributors have commented on others' pull requests, this finding indicates that *Peripheral* contributors largely consist of project users who identify issues or request specific features based on their use of the ML library. Their involvement in pull request discussions often revolves around collaborating with core members to implement desired changes, reflecting a user-driven engagement with the project.

5.2.3.2 Work Preferences

As shown in Table 7, the null hypothesis H_{07} is rejected for all work preference features. This indicates that *Core* and *Peripheral* contributors have significantly different work preferences, in terms of their contribution dynamics and the level of balance of their contributions across various types of OSS activities.

Core contributors exhibit more complex contribution dynamics compared to *Peripheral* contributors. Contribution dynamics of *Core* contributors, measured by *binmed_entropy*, *c3(1)*, *c3(2)*, and *c3(3)*, exhibit significantly higher complexity than *Peripheral* contributors, indicating greater fluctuations in their period OSS activity time series. Moreover, *Core* contributors have significantly more peaks (i.e., *number_cwt_peaks*), a higher ratio of continuous time above mean (i.e., *longest_strike_above_mean*), and a lower ratio of the continuous time below mean (i.e., *longest_strike_below_mean*) in their OSS activities time series, compared to *Peripheral* contributors. This indicates that **Core contributors tend to make more peak contributions within a period and engage in longer times of continuous intensive contributions, while *Peripheral* contributors tend to maintain constant contribution levels and have longer continuous inactive time.**

Core contributors make less balanced contributions than *Peripheral* contributors. As shown in Table 7, *Core* contributors are significantly higher than *Peripheral* contributors on *diverse* with a small effect size and significantly lower on *balance* with a medium effect size. **This indicates that Core contributors tend to make more diverse**

Table 8. Mann-Whitney U test and effect size results for contributor technical importance.

Technical Importance	Group1: Core Group2: Peripheral	Group1: Core-Nighttime Group2: Core-Daytime	Group1: Peripheral-Nighttime Group2: Peripheral-Daytime
max_period_centrality	medium (0.473)	negligible (-0.094)	not significant
max_centrality_period	medium (0.467)	negligible (-0.089)	not significant
max_commit_centrality	medium (0.452)	negligible (-0.078)	not significant
max_centrality_day	medium (0.46)	not significant	negligible (0.045)

Three columns from left to right correspond to the null hypothesis H_{010} , H_{011} , and H_{012} respectively.

contributions across various types of OSS activities with a stronger focus on one or few types of OSS activities, while *Peripheral* contributors make more even amount of contributions across fewer types of OSS activities.

The work preference differences between *Daytime* and *Nighttime* subgroups are negligible. The null hypothesis H_{08} is rejected for 8 (out of 9) work preference features, indicating that compared to *Core-Nighttime* contributors, *Core-Daytime* contributors tend to have more complex contribution dynamics, and make more diverse and less balanced contributions across different types of OSS activities. However, the differences are negligible in practice. Similarly, the null hypothesis H_{09} is also rejected for 8 work preference features, with negligible effect sizes. This shows that *Peripheral-Nighttime* contributors tend to have more complex contribution dynamics than *Peripheral-Daytime* contributors, but the differences are as well negligible in practice.

5.2.3.3 Technical Importance

Core contributors tend to achieve a higher maximum technical importance within a project than *Peripheral* contributors. As shown in Table 8, the null hypothesis H_{010} is rejected for all technical importance metrics with medium effect sizes. *Core* contributors have significantly higher maximum technical importance than *Peripheral* contributors, both at the commit-level (i.e., *max_commit_centrality*) and period-level (i.e., *max_period_centrality*). This means that when comparing the importance of individual commits, which considers the average influence of files changed within the commit, the most important commit made by *Core* contributors tends to have higher importance (i.e., *commit_centrality*) than the most important commit made by *Peripheral* contributors. Similarly, the period of the highest technical importance for *Core* contributors, which considers both the importance and intensity of commits made within a period, tends to have higher importance (i.e., *period_centrality*) than those for *Peripheral* contributors. Therefore, *Core* contributors tend to reach higher technical importance within a project by making intensive changes to central files, which have a greater impact on the codebase and core functionalities. *Peripheral* contributors tend to stay with modifying peripheral files that have less impact on the system and are often confined to very specific functions, such as updating documentation or fixing a parameter in an API. An example illustrating commit contributions of different levels of technical importance can be found in Figure 12 in Appendix D, where commits with high and low centrality are made to core kernel functions and TensorFlow Lite demo code, respectively.

Core contributors tend to require a longer time to progress to their maximum technical importance than *Peripheral* contributors. As shown in Table 8, *Core* contributors are significantly higher than *Peripheral* contributors in terms of both the number of days before making their most important commit (i.e., *max_centrality_day*) and the number of periods to reach the period of their highest technical importance (i.e., *max_centrality_period*), indicating they require a longer time to reach their maximum technical importance within a project. *Core-Nighttime* and *Core-Daytime* contributors have a median of 100.5 and 126 days, respectively, to make their most important commit. For both *Core* profiles, contributors take a median of 2 periods to reach the period of the highest technical importance. **This suggests a**

Table 9. Distribution of ML-specific contributions across contributor profiles.

Category of Contribution	Core-Nighttime	Core-Daytime	Peripheral-Nighttime	Peripheral-Daytime
User-Level API	8.3%	16.3%	27.1%	37.5%
Graph-Level Implementation	11.5%	17.4%	14.6%	9.4%
Operation Implementation	29.2%	18.5%	17.7%	16.7%
General Utility	11.5%	5.4%	4.2%	6.3%
Environmental-Dependent Processing	19.8%	13%	7.3%	10.4%
General Software Development	19.8%	29.3%	29.2%	19.8%

progression in technical importance for *Core* contributors within the first six months of activity, followed by a shift away from highly technical and intensive contributions. In contrast, *Peripheral-Nighttime* and *Peripheral-Daytime* contributors have a median of reaching the period of the highest technical importance in 1 period and making the most important commit within 6 and 4 days respectively, indicating that the technical importance of *Peripheral* contributors tends to remain the same as they join the project. The *Peripheral* contributors do not make significant progression over time.

The differences in technical importance between the Daytime and Nighttime subgroups are negligible. The null hypothesis H_{011} is rejected for the technical importance metrics *max_period_centrality*, *max_centrality_period*, and *max_commit_centrality*, all with negligible effect sizes. This means that *Core-Daytime* contributors tend to achieve a significantly higher maximum level of technical importance and take significantly longer to reach this maximum compared to *Core-Nighttime* contributors. However, the effect sizes are negligible, meaning these differences, though statistically significant, are negligible in practice. The null hypothesis H_{012} is only rejected for *max_centrality_day* with negligible effect sizes. This indicates that while both *Peripheral-Daytime* and *Peripheral-Nighttime* contributors reach similar levels of maximum technical importance, *Peripheral-Nighttime* contributors tend to take significantly longer to make their most important commit compared to their *Daytime* counterparts, though this difference is also practically negligible.

5.2.3.4 ML-Specific Contributions

Based on our Chi-Squared test and Cramér's V results, we reject the null hypothesis H_{013} with a large effect size, while accepting H_{014} and H_{015} . The test results indicate significant differences in the distribution of ML-specific contributions between *Core* and *Peripheral* contributors. The differences between their *Daytime* and *Nighttime* subgroups are insignificant.

Operation implementation is the most frequent ML-specific contribution for Core contributors. As shown in Table 9, *Core-Nighttime* contributors have the highest percentage of contributions to operation implementations (29.2%), which are core functionalities of the deep learning framework and often involve complex, DL-specific algorithms. This highlights their expertise and focus on the more complex technical aspects of the project. This finding aligns with previous observations that *Core-Nighttime* contributors tend to make more complex code contributions, which require extensive discussion and result in larger commits. *Core-Daytime* contributors demonstrate a more balanced distribution across different ML-specific contributions, indicating that they often take on diverse responsibilities, ranging from implementing core ML operations to graph-level optimizations and supporting general project development. This broad involvement indicates that these contributors are likely to be dedicated project maintainers who possess the

knowledge and contribute across multiple layers of the ML framework. As stated in the PyTorch contributor guide¹⁹, once a pull request is merged, maintainers must be capable of maintaining the corresponding code. Notably, 65% Pytorch maintainers (as listed in the persons of interests²⁰) identified in our dataset are categorized to the *Core-Daytime* profile.

User-level API is the most frequent ML-specific contribution for *Peripheral* contributors, where 27.1%-37.5% of their pull requests fall into the User-Level API category. These pull requests focus on making changes to APIs related to data processing, model building, model deployment, and utility functions. This indicates their role as users of the ML libraries who observe issues or have demands when using the ML library through the APIs and therefore make code contributions to solve their needs. Therefore, their contributions mainly improve the usability and accessibility of these projects.

Contributions to general utility and environmental-dependent processing are relatively low across all profiles, ranging from 4.2% to 11.5% and 7.3% to 19.8% respectively. This can be attributed to the fact that these components take up only a small portion of the project's source code and require less maintenance effort. Additionally, general utilities involve common functions and data structures that are highly reusable across different components of the project, thereby reducing the need for frequent updates. Environmental-dependent processing contributions often require specialized expertise related to specific hardware devices or software environments. Therefore, only a limited number of contributors possess the necessary knowledge to make these changes. For example, third-party contributors from companies like Intel or AMD might contribute to ensure compatibility with their new products or introduce new CUDA optimization techniques to improve performance. Notably, *Core-Nighttime* contributors make more contributions to environmental-dependent processing than the other three profiles (i.e., 19.8%), indicating their higher involvement in these tasks that tend to be more complex.

General software development contributions are prevalent among all four contributor profiles, accounting for 19.8% to 29.3% of their pull requests. This indicates that, similar to traditional OSS projects, contributors in ML projects invest considerable effort in maintaining general software infrastructure and supporting non-ML-specific tasks, regardless of their profiles. These tasks include updating CI/CD scripts, refactoring code, testing, enhancing graphical user interfaces, updating documentation, and other general development activities. Such efforts are important for streamlining the software development workflow and also ensuring the quality and effective integration of ML-specific contributions into the project.

RQ2 Summary

The most common workload composition pattern among all contributor profiles is Committer. *Core* contributors demonstrate more complex contribution dynamics and make less balanced contributions compared to *Peripheral* contributors. *Core* contributors also tend to achieve higher maximum technical importance and exhibit a progression towards increasing technical importance within the first 2 active periods, whereas *Peripheral* contributors typically have the highest technical importance when initially joining the project. The most frequent ML-specific contribution is operation implementation for *Core* contributors, while contributions to user-level API are the most frequent for *Peripheral* contributors.

5.3 RQ3: Which factors of contributor OSS engagement are related to a project's popularity growth

5.3.1 Motivation

¹⁹https://pytorch.org/docs/stable/community/contribution_guide.html

²⁰https://pytorch.org/docs/main/community/persons_of_interest.html

The diverse distribution of contributors with varying OSS engagements results in varied project dynamics. Analogous to how demographic diversity influences a nation's growth and stability, the distribution of contributors with different workload compositions and work preferences can significantly impact the success and vitality of an OSS project. This becomes particularly valuable during periods of significant turnover, enabling project maintainers to quickly recognize the types of contributions (e.g., issue reporting, issue discussion, committing, pull request discussion, and code review) from departing contributors and assess the potential impact on various project activities in their absence. In this research question, we investigate the correlation of the diversity of contributors with different workload compositions and work preferences with the project popularity, in terms of the increase in star ratings and the number of forks.

5.3.2 Approach

We build four mixed-effect models to explore how contributor workload composition patterns and work preferences are correlated with the increase of project stars and forks. Our detailed approach is outlined as follows:

Independent variable Preparation: We prepare two sets of independent variables: workload composition pattern-related variables and work preference-related variables as listed in Table 10.

- **Workload composition pattern related-variables:** For each period (as defined in Section 4.1) within a project, we calculate the ratio of active contributors belonging to each of the five contributor workload composition patterns, and it denotes as $\langle Pattern_name \rangle_ratio$. For example, $Issue_Reporter_ratio$ circulates the proportion of contributors belonging to the Issue Reporter pattern among all active contributors in that period. This yields five variables describing the distribution of contributors of different workload composition patterns during that specific period.

We also compute the ratio of the five types of OSS contributions (i.e., issue reports, issue comments, commits, pull request comments, and code reviews) made by contributors belonging to each workload composition pattern. We denote it as $\langle Pattern_name \rangle_ \langle contribution_type \rangle_ratio$. For example, $Issue_Reporter_issue_ratio$ denotes the proportion of issue reports raised by contributors belonging to the Issue Reporter pattern among all issue reports raised in that period. Therefore, 25 variables are extracted to capture the diversity of contributions from contributors of five workload composition patterns at that period.

We add a variable *period* to account for the influence of different developmental stages of software projects on their popularity. In total, 31 workload composition pattern related-variables are extracted. We apply the Spearman Rank Correlation test to measure the variable correlations and remove one from each pair of highly correlated features with a coefficient greater than 0.7 [76]. 13 non-correlated variables remain, as shown in Table 10.

- **Workload preference related-variables:** For each period in a project, we calculate the median value of each work preference feature (identified in Section 4.2) among the active contributors within that period. A median value represents the collective contributor work preference during that period. As a result, we obtain 9 variables corresponding to the respective median value of each work preference feature. We compute the average contribution made by active contributors to each of the five types of OSS activities within the period, to represent the project-level preference for different types of OSS activities. As previously mentioned, we include a *period* variable to account for the impact of project development stages on the project popularity. In total, we obtain 15 work preference-related variables. We apply the Spearman Rank Correlation test to identify and remove 5 correlated variables. 10 remaining variables are shown in Table 10.

Dependent Variable Preparation: We use the number of stars and forks as project popularity metrics. More specifically, the number of forks reflects the interest in a project from potential contributors, while the number of stars indicates interest from both potential users and contributors. We use Github GraphQL API²¹ to collect the timestamp of each star received by a project and use Github Forks API²² to collect the timestamp when each fork occurs. For each project, we count the number of stars and forks gained within each period (i.e., 90 days period as described in Section 4.1) and use the increases of stars and forks in consecutive periods as our dependent variables. Due to variations in popularity levels among the subject projects, the counts of stars and forks may be at different scales for different projects. Using the counts of stars and forks directly as dependent variables can skew the model results. Therefore, we apply Min-Max Normalization to scale the star and fork counts for different periods within each project to a range of 0 to 1. The formula of Min-Max Normalization is presented in Equation 1, where features are the subject projects and i indexes the periods of the projects. The normalized stars and forks serve as our dependent variables.

Model Construction: We construct four mixed-effect models to identify the correlation between (i) contributor workload composition pattern related-variables and the increase in star ratings, (ii) workload composition pattern related-variables and the increase in forks, (iii) work preference related-variables and the increase in star ratings, and (iv) work preference related-variables and the increase in forks respectively.

Mixed-effects models are statistical regression models that integrate both fixed and random effects [82]. Fixed effects are variables with consistent slopes and intercepts across all data points, while random effects account for variations between experimental groups, such as projects in our case. Mixed-effect models can estimate different slopes or intercepts for different experimental groups. Our mixed-effect models use projects as a grouping factor and assume different intercepts for different projects. We construct the mixed-effect models using Python package *statsmodels* [85]. Note that we exclude the initial periods for each project, where no forks are recorded (i.e., first 19, 13, 2, and 1 periods for Pytorch, Theano, scikit-learn, and MXNet respectively), due to a concern of these repositories being private during those times. Similarly, the first period for every project is excluded from the model training, because we find that for all projects, the first period has significantly more stars and forks than other periods, which could skew the model results.

Model Explanatory Power Evaluation: We use conditional R^2 and marginal R^2 metrics to evaluate the fit of our mixed-effect models. Marginal R^2 describes the proportion of variance explained by fixed effects. Conditional R^2 describes the proportion of variance explained by both fixed effects and random effects [73].

Model Result Analysis: To understand the important features in the mixed-effect models, we examine the z-values and their corresponding p-values for each independent variable in the model result. z-value is associated with the t-statistic which tests the significance of each fixed effect coefficient, and it is more reliable than Wald Statistics with limited data points [9]. A fixed effect is significant if it has $\Pr(>|z|) < 0.05$, and such variables are marked with asterisks in Table 10. We also analyze the coefficient sign of each independent variable. Positive coefficients, denoted by upward arrows ($\nearrow$) in Table 10, suggest a positive correlation with the dependent variables (i.e., stars or forks), whereas negative coefficients, indicated by downward arrows ($\searrow$), imply a negative correlation.

5.3.3 Results

The mixed-effect models of modeling star increase with work preference features have the highest explanatory power. Our mixed-effect model for analyzing work preferences and star increase has both conditional R^2 and marginal R^2 of 0.64, and the model for work preferences and fork increase has a conditional R^2 and marginal R^2 of both

²¹<https://docs.github.com/en/graphql>

²²<https://docs.github.com/en/rest/repos/forks>

Table 10. Mixed-effects models results.

Work Preference	Star		Fork	
	Signif.	Rel.	Signif.	Rel.
period	***	↘	***	↘
binned_entropy	**	↘	**	↘
c3_1		↗	.	↗
diverse		↗		↗
balance	***	↗	***	↗
commit	***	↘	***	↘
issue	**	↗		↗
issue_comment		↘		↗
pr_comment		↘		↗
review	***	↗	***	↗
Workload Composition Pattern				
Issue_Reporter_ratio	*	↗	*	↗
Issue_Discussant_ratio	***	↗	***	↗
Committer_ratio		↗	.	↗
Issue_Reporter_issue_ratio		↗		↗
Issue_Discussant_issue_comment_ratio	**	↘	**	↘
Issue_Discussant_review_ratio	*	↗	.	↗
Committer_issue_ratio	*	↘		↘
Committer_review_ratio	.	↗		↗
PR_Discussant_commit_ratio	*	↘	.	↘
PR_Discussant_issue_ratio		↗		↗
PR_Discussant_review_ratio	***	↗	***	↗
Code_Reviewer_issue_ratio		↗	.	↗
Code_Reviewer_review_ratio	.	↗	*	↗
Signif. codes: Pr(> z) = 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1				
Rel.: ↗ stands for a positive association between the independent and dependent variables, and ↘ stands for a negative association.				

0.54. This result shows that both models have good explanatory power and most of the variances are explained by the fixed effects. The mixed-effect model of workload composition patterns with star increase has both conditional R^2 and marginal R^2 of 0.31, and the model of fork increase has both conditional R^2 and marginal R^2 of 0.26, indicating that both models have all the variances explained by the fixed effects. We emphasize that our results reflect correlations rather than causal relations; thus, causality should not be inferred from these observations.

5.3.3.1 Important Work Preference Features to Project Popularity

Work preference features such as the balance of contribution and the average contributor code reviews have significant positive correlation with project popularity in terms of both star rating and the number of forks. Table 10 presents the variable importance and coefficients of our mixed effect models. We observe that more balanced contributions across various types of OSS activities (i.e., *balance*) and a higher number of average code reviews conducted by each contributor (i.e., *review*) have a significant positive correlation with increases in both stars and forks. This correlation indicates that when contributors distribute their efforts evenly across various OSS activities or engage in more code reviews, the likelihood of attracting new users or contributors tends to rise.

The developmental stages of a project (i.e., *period*) and the average number of commits per contributor (i.e., *commit*) are negatively associated with star and fork increases. This indicates a reduced likelihood of attracting more new users and contributors as project development progresses to later stages and when contributors are heavily taken on committing responsibilities. The complexity of contributors' contribution dynamics (i.e., *binned_entropy*) is also negatively correlated with star and fork increases, indicating an increased likelihood of attracting potential contributors when existing contributors have constant and less random contribution frequencies. The average number of issue reports raised per contributor (i.e., *issue*) is positively associated with star increases. As issue reporting is often an accessible entry point for newcomers²³, the active participation of current contributors in the issue discussions (i.e., *Issue_Discussants_ratio*) could help newcomers efficiently resolve their problems or convert their ideas into appropriate code contributions. Potential users and contributors may see the project as responsive and supportive, then star the project.

5.3.3.2 Important Workload Composition Pattern Features to Project Popularity

Workload composition pattern features, such as the ratio of Issue Reporters and Issue Discussants among all contributors, and the ratio of code reviews made by PR Discussants, have significant positive correlation with project popularity in terms of both star rating and the number of forks. As shown in Table 10, the ratio of Issue Reporters (i.e., *Issue_Reporter_ratio*) and Issue Discussants (i.e., *Issue_Discussants_ratio*) among all contributors in the project has a significant positive correlation with the increase in both stars and forks. This suggests that projects are more likely to gain popularity when a higher proportion of contributors focus on reporting issues or requesting new features, and when more contributors are actively involved in discussing these issues and actively responding to questions or requests from users or potential contributors. Additionally, PR Discussants focus on participating in pull request discussions, often guiding other contributors to revise and align their changes with project requirements. Their familiarity with proposed changes could make their subsequent code reviews more reliable, which potentially improves the quality of the codebase and the project popularity. Furthermore, when newcomers submit pull requests, receive guidance from experienced contributors, and ultimately have their contributions approved by them, they are likely to feel welcomed, supported, and acknowledged for their efforts.

A higher ratio of code reviews conducted by Issue Discussants (i.e., *Issue_Discussant_review_ratio*) has a significant positive correlation with the increase in star ratings. This association indicates that projects tend to gain more star ratings when a larger portion of code reviews are made by Issue Discussants. A possible explanation is that Issue Discussants are familiar with ongoing project issues and feature requests through their active participation in the issue discussions. Therefore, their code reviews are particularly effective in assisting contributors to resolve their issues or implement features in a way that aligns with the immediate needs of the project. This interaction could be especially beneficial for retaining newcomers, as Issue Discussants may already engage with newcomers during issue discussions before helping them with code reviews. When newcomers begin their journey in the project by discussing their issues with existing contributors and eventually making successful pull requests with their guidance, they are likely to feel welcomed and accepted within the project community. A real-world example is shown in Figure 11 in Appendix D.

A higher proportion of code reviews conducted by Code Reviewers is significantly positively correlated with the increase in forks. Table 10 shows that the proportion of code reviews made by Code Reviewers (i.e., *Code_Reviewer_review_ratio*) have a significant positive correlation with fork increases but not with stars. This observation can be explained by the fact that, while both users and potential contributors star projects to express their

²³<https://github.com/pytorch/ignite/blob/master/CONTRIBUTING.md>

interest, individuals who fork projects often have the intention of starting to make code contributions. Therefore, the code reviews provided by Code Reviewers hold greater significance for potential contributors than users.

RQ3 Summary

Contributors making balanced contributions across various OSS activities and a higher average number of code reviews are significantly correlated with the increase in project popularity in terms of star ratings and forks. Moreover, a higher proportion of Issue reporters and Issue Discussants among existing contributors and a larger portion of code reviews made by PR Discussants are significantly correlated with greater increases in star ratings and forks. The proportion of code reviews conducted by Issue Discussants is significantly correlated with star ratings. The proportion of code reviews made by Code Reviewers is significantly correlated with the increase in forks.

5.4 RQ4: How does contributor OSS engagement evolve?

5.4.1 Motivation

From our observation of contributor engagement within a project in RQ2, we notice that their OSS engagements are not static; they shift over time. This motivates us to explore how the workload composition of contributors across different profiles changes as they spend more time on a project, as well as how the demand for contributors with diverse workload compositions changes as project development progresses. Moreover, we are also interested in understanding how contributors' work preferences and technical importance shift as they stay longer in the project and make more contributions. To provide insights into contributors' behavioral trajectories, we aim to identify if there are significant trends in the evolution of contributors' workload composition, work preferences, and technical importance. Recognizing such trends may assist project maintainers in better comprehending the dynamics of potential long-term contributors.

5.4.2 Approach

We explore the trends in the evolution of workload compositions, work preferences, and technical importance. Our detailed approach is described as follows:

5.4.2.1 Evolution of Workload Composition

To identify the evolution of a contributor's workload composition, we first extract the contributor's active *periods* (as described in Section 4.1), during which the contributor has made OSS contributions to the project. We then extract the workload composition pattern associated with the contributor in each period, thus obtaining a time series of workload composition patterns that reflects the contributor's workload compositions over time. We encode the workload composition patterns into numerical values, with Issue Reporter represented as 1, Issue Discussant as 2, Committer as 3, PR Discussants as 4, and Code Reviewer as 5. This numerical representation aligns intuitively with the level of expertise needed to handle the respective workload.

Cox-Stuart trend test [21] is a statistical method to detect the presence of a monotonic trend in time series. It is well-suited for identifying whether a sequence exhibits a significant upward or downward trend without assuming any specific distribution. We employ the Cox-Stuart trend test from the R package *randtests*²⁴ to identify if a significant increasing or decreasing trend exists in our numerical workload composition time series. For each time series, we conduct both right-sided and left-sided Cox-Stuart trend tests: the right-sided test tests the null hypothesis H_{016} that there is no significant increasing trend in the given sequence, while the left-tailed test tests the null hypothesis H_{017}

²⁴<https://cran.r-project.org/web/packages/randtests/index.html>

that there is no significant decreasing trend. Both tests are conducted with a 95% confidence level. We test against these two null hypotheses for all the Cox-Stuart trend tests in the rest of the paper.

- H_{016} : *There is no significant increasing trend in the given time series/sequence.*
- H_{017} : *There is no significant decreasing trend in the given time series/sequence.*

When applying the Cox-Stuart trend test on workload composition time series, we exclude the contributors who have less than 2 active periods, as the test requires at least two datapoints in the time series.

To pinpoint how long-term contributors' workload compositions change across different stages of their project involvement, we divide their workload composition time series into three subsequences, representing their early, middle, and late stages, following a similar approach to Noei et al.[77]. We exclude very short-term contributors with fewer than three active periods, as they represent a large base of one-time and infrequent contributors whose behavior may introduce noise rather than reflecting meaningful evolution patterns. For the remaining contributors, we calculate the quartiles of their active periods to define project involvement stages: 1) early stage: one's first active period to the 5th active period (i.e., the first quartile); 2) middle stage: one's 6th active period to the 10th active period (i.e., the third quartile); 3) late stage: the 11th active period onward. We study the long-term contributors who have entered the late stage by applying the Cox-Stuart trend test on the jointed sequence of the early and middle stages to examine the changes from the early to middle stage, and the same for the middle to late stage.

Additionally, we examine the evolution of both the prevalence and distribution of each workload composition pattern within the subject projects, to understand the sustainability and the demand for contributors with particular workload compositions across different stages of project development. For each project, we count the number of contributors belonging to a workload composition pattern in each period into a time series to analyze the prevalence of the pattern as project development progresses. Similarly, we build another time series for the ratio of contributors belonging to a workload composition pattern among all contributors in each period to analyze how the distribution of the pattern evolves within a project. We apply the Cox-Stuart trend test to examine the trends in the time series of counts and ratios of contributors belonging to each workload composition pattern. To further pinpoint the major changes in contributor workload composition at different project development stages, we divide each time series into three equal-length subsequences, representing the early, middle, and late stages of project development. We apply the Cox-Stuart trend test on the jointed sequence of the early and middle stages to identify the change of contributor workload compositions in a project, from its early to middle stages, and the same for the middle to late stage.

5.4.2.2 Evolution of Work Preferences

To identify trends in the evolution of contributors' work preferences, our approach is similar to identifying the evolution of workload composition in Section 5.4.2.1. Figure 7 illustrates our detailed approach. For each contributor, we first extract the periods where the contributor is active in the project. We extract nine work preference features (as described in Section 4.2) from the contributor's OSS activity time series within each active period. We build a time series for each work preference feature, representing its evolution across the contributor's active periods. Therefore, we obtain nine work preference feature time series for each contributor to analyze the evolution of their work preferences. We apply the same procedure as in Section 5.4.2.1 to select contributors and examine the trends in their work preference feature time series, as well as the trends in the early-to-middle-stage and middle-to-late-stage subsequences.

5.4.2.3 Evolution of Technical Importance

We study the evolution of contributors' technical importance at two levels of granularity: per commit and per period. Analyzing contributors' technical importance at the commit level allows us to understand the shift in the importance

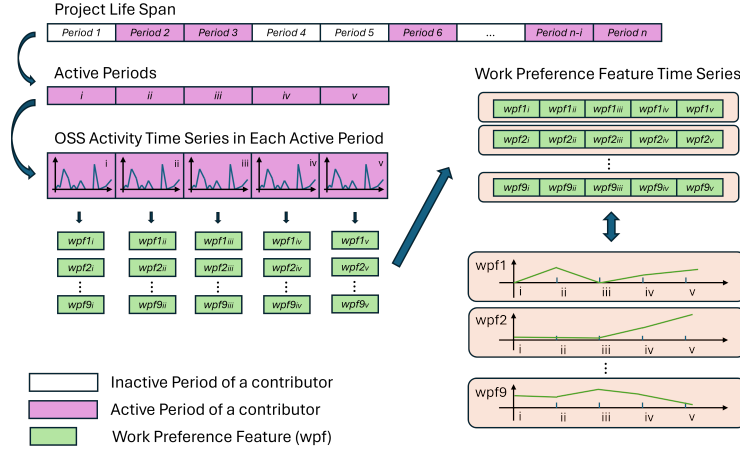


Fig. 7. Example of building a contributor's work preference feature time series.

of their code changes more precisely, such as from making changes to the very specific functions in the peripheral files to modifying the influential files that have a greater impact on the system, or the opposite. Moreover, period-level evolution reflects contributors' changes in technical importance at a higher level, considering both the importance and the intensity of their code contribution over time.

To identify the trend of contributors' commit importance, we extract all commits made by a contributor and sort them into chronological order. We calculate the *commit centrality* (as described in Section 4.3) of each commit, thereby obtaining a sequence of commit centrality that reflects the importance of each commit made by a contributor over time. We apply the same procedure as in Section 5.4.2.1 to select contributors and examine the trends in their commit centrality sequence, as well as the trends in the early-to-middle-stage and middle-to-late-stage subsequences.

Similarly, to identify the trend of contributors' period technical importance, we extract the contributors' active periods, in which a contributor has made OSS contributions to the project. We calculate the *period centrality* (as described in Section 4.3) of each active period and construct a time series of *period centrality* for each contributor. We examine the trends in their period centrality time series and the trends in the early-to-middle-stage and middle-to-late-stage subsequences.

5.4.3 Results

Table 14 shows the overall evolution trends of workload composition, work preference, and technical importance for contributors in each profile. Table 15 presents the early-to-middle-stage and middle-to-late-stage trends. In these tables, the upward arrows denote rejection of the null hypothesis H_{016} , indicating increasing trends in the given time series, with the percentages indicating the proportion of contributors within the profile exhibiting increasing trends. Likewise, downward arrows represent the rejection of the null hypothesis H_{017} and decreasing trends.

5.4.3.1 Evolution of Workload Composition

A small portion of long-term contributors exhibit significant trends in their workload composition towards handling tasks requiring higher expertise, while the majority of contributors do not exhibit a significant trend in shifting workload composition. As shown in Table 14, approximately 3% *Core* contributors and 0.3% *Peripheral* contributors demonstrate significant trends in shifting workload composition. We manually investigate

these contributors and observe that they are all long-term contributors with at least 10 active periods (i.e., being active in the subject project for more than 2.5 years). Among these contributors, 78% have an increasing trend in their workload composition, which means that their workload composition evolves in the direction of Issue Reporters → Issue Discussants → Committer → PR Discussant → Code Reviewer. This indicates that these long-term contributors tend to shift their focus toward the OSS tasks that are typically considered to require higher expertise. Table 15 shows that the increasing trends of workload composition for long-term contributors primarily occur during their early to middle stage of project involvement. Approximately 1.4% *Core* contributors exhibit an increasing trend in workload composition during this stage, while only 0.35% experience a decreasing trend. In contrast, a similar proportion of long-term contributors show increasing and decreasing trends in workload composition during their middle to late stages of involvement.

The Committer’s and Code Reviewer’s contributions are more sustainable compared with other roles as ML project development progresses. Table 11 shows the trends in the prevalence (i.e. count) and the proportion (i.e., ratio) of contributors exhibiting each workload composition pattern within the subject projects. In all projects, both in their early stages and overall trends, the number of Committers and their ratio within the project either remains constant or demonstrates increasing trends. This shows the enduring popularity and community engagement of the selected ML projects. However, in the late stages, MXNet and Theano/Aesara exhibit a decreasing trend in the number of Committers, possibly due to the delaying popularity of these projects. Seven (out of eight) projects show either an increasing trend or a stable ratio of Code Reviewers, indicating the sustained demand for code reviews throughout the ML project development stages. Additionally, five projects exhibit an increasing trend in the count and ratio of Code Reviewers in their early to middle stages, indicating significant progression of contributors to Code Reviewers during this period, with only a few contributors eligible for code review in the early stage.

The Issue Reporter’s and Issue Discussant’s contributions are less sustainable compared with other workload composition patterns as ML project development progresses. As shown in Table 11, 4 (out of 8) projects exhibit a decreasing trend in the prevalence of Issue Reporters and Issue Discussants. This trend likely stems from the emerging phase of the ML library. When a new ML library emerges, there is often a surge of interest due to the growing popularity of ML in recent years. For instance, we observe that TensorFlow, Keras, MXNet, and ONNX reach the peak number of Issue Reporters within 1 to 2 years after they open-sourced. During this time, OSS community members are eager to explore the new ML library and actively provide feedback, report bugs, and request new features. After this initial interest, enthusiastic contributors may shift their focus toward code contributions possibly transitioning to roles like Committers, while others may depart from the project.

5.4.3.2 Evolution of Work Preferences

A portion of long-term contributors show a tendency towards fewer, constant, and balanced contributions, while the majority of contributors do not demonstrate a clear trend in shifting work preferences. Similar to our finding on workload compositions, significant trends in work preferences are observed only among long-term contributors (primarily *Core* contributors). For the contributors with significant changes in each work preference feature, we observe that 69% to 81% of them have a decreasing trend for the complexity of contribution dynamics (i.e., *binned_entropy* and *C3 statistics*), the number of peak contributions (i.e., *number_cwt_peaks*), and the time with continuous intensive contribution (i.e., *longest_strike_above_mean*). This suggests that these long-term contributors tend to reduce intensive contributions and progress towards making constant contributions. Moreover, among contributors who have significant changes in their contribution balance, we observe 79% of them have increasing trends. This

Table 11. Project-level trend of workload composition.

Project Stages	Project	Issue Reporter Count Ratio	Issue Discussant Count Ratio	Committer Count Ratio	PR Discussant Count Ratio	Code Reviewer Count Ratio
Early-Middle Trend	tensorflow	- (***)↘	- (***)↘	(***)↗ (***)↗	- -	- -
	pytorch	- -	(*)↗ -	(*)↗ -	- -	(**)↗ (***)↗
	scikit-learn	(***)↗ -	(***)↗ -	(***)↗ -	(***)↗ (*)↘	(***)↗ (***)↗
	keras	(**)↘ (***)↘	(*)↘ -	- -	- -	- -
	mxnet	(***)↘ (***)↘	- -	- -	(*)↗ (***)↗	(**)↗ (***)↗
	theano_aesara	- -	- -	- -	- -	(***)↗ (***)↗
	onnx	- -	- -	- -	- -	(*)↘ -
Middle-Late Trend	tensorflow	(***)↘ (***)↘	(***)↘ (***)↘	- (***)↗	(*)↘ -	- -
	pytorch	(*)↗ -	- -	- (*)↘	- -	(**)↗ -
	scikit-learn	- -	- -	(*)↗ (*)↗	- (*)↘	- -
	keras	- -	- (*)↘	(*)↗ (***)↗	- (*)↘	- -
	mxnet	(***)↘ (***)↘	(***)↘ -	(***)↘ (*)↘	(***)↘ -	(***)↘ -
	theano_aesara	- -	- (*)↗	(**)↘ (***)↘	- -	- -
	onnx	- -	- -	- -	- -	- -
Overall Trend	tensorflow	(*)↘ (***)↘	(*)↘ (***)↘	(***)↗ (***)↗	- -	- (*)↘
	pytorch	(*)↗ -	(*)↗ -	(***)↗ (***)↘	(*)↗ (*)↗	(***)↗ (***)↗
	scikit-learn	(***)↗ -	(***)↗ -	(***)↗ -	(*)↗ (***)↘	(***)↗ (***)↗
	keras	(***)↘ (***)↘	(***)↘ -	- (*)↗	- (*)↗	- -
	mxnet	(***)↘ (***)↘	(***)↘ -	(***)↘ (*)↘	- (*)↗	- (***)↗
	theano_aesara	(***)↘ -	(***)↘ -	(***)↘ (***)↘	(***)↘ -	(***)↗ (***)↗
	onnx	- -	- -	- -	- -	(***)↘ -
Overall Trend	tensorflow	- -	- (*)↘	(***)↗ (***)↗	- -	- (***)↗
	pytorch	- -	- (*)↘	(***)↗ (***)↘	- -	- (***)↗
	scikit-learn	- -	- (*)↘	(***)↗ (***)↘	- -	- (***)↗
	keras	- -	- (*)↘	(***)↗ (***)↘	- -	- (***)↗
	mxnet	- -	- (*)↘	(***)↗ (***)↘	- -	- (***)↗
	theano_aesara	- -	- (*)↘	(***)↗ (***)↘	- -	- (***)↗
	onnx	- -	- (*)↘	(***)↗ (***)↘	- -	- (***)↗

↗ : The null hypothesis H_{016} is rejected and the given time series has an increasing trend.↘ : The null hypothesis H_{017} is rejected and the given time series has a decreasing trend.- : Both H_{016} and H_{017} are accepted. There is no significant trend in the given time series.

Confidence Level: (***) 99.9%, (**) 99%, (*) 95%.

indicates that long-term contributors tend to progress towards allocating their contributions more evenly across various OSS activities. We also observe a greater proportion of contributors have decreasing trends for the number of commits, issues, issue comments, and pull request comments, while having increasing trends for code reviews. This suggests a shift towards undertaking more review duties and reducing contributions to other OSS tasks. Table 15 shows that significant shifts of work preferences towards increasing contribution intensity primarily occur in the early to middle stages for the long-term contributors, while shifts of work preferences towards decreasing contribution intensity primarily occur in their middle to late stages.

5.4.3.3 Evolution of Technical Importance

A portion of long-term contributors progress towards reduced technical importance, while the majority of contributors do not demonstrate a clear trend in shifting technical importance. We observe that approximately 8% *Core* contributors exhibit significant increasing trends in the evolution of their commit importance (i.e., commit_centrality), and 12% show significant decreasing trends. This indicates two different evolution patterns in the commit importance (i.e., commit_centrality): a gradual increase in the importance of their code contributions or a shift away from highly technical code contributions that modify the influential files in the repository. Period technical importance (i.e., period_centrality) measures both the volume and the importance of the commits made within a period. Similar to our observations in the evolution of workload compositions, significant trends in period technical importance

are observed only among long-term contributors who have been active in the projects for over 2.5 years. Among these contributors with significant changes in their technical importance, we observe a predominant trend toward decreasing technical importance. This finding implies a tendency of long-term contributors to shift away from intensively making highly technical code contributions. Table 15 shows that a similar proportion of long-term contributors have decreasing period technical importance in their early to middle stage and the middle to later stages.

RQ4 Summary

The Committer’s and Code Reviewer’s contributions are more sustainable as project development progresses, while the Issue Reporter’s and Issue Discussant’s contributions are less sustainable. A portion of long-term contributors show a tendency towards fewer, constant, and balanced contributions, and shift away from highly technical and intensive contributions. Nevertheless, the majority of contributors do not exhibit a significant trend in their evolving OSS engagement.

6 Discussion

In this section, we extend and provide more discussions on the findings from our research questions, to provide practical suggestions for ML contributors.

Starting with contributing to ML user-level APIs and avoiding complex changes increases acceptance rates for ML newcomers. Our findings in RQ2 recommend that ML newcomers contribute to user-level APIs, such as data processing or model building and deployment, and prioritize simpler tasks like refactoring or test updates. These contributions are more likely to be accepted and help newcomers establish themselves in the community. While encouraging newcomers to start with simpler issues aligns with well-established practices in OSS management, such as the “good first issue” guideline, our findings reinforce their continued importance within the ML OSS projects. Complex changes, such as adding new ML operators involving complicated mathematics, often face higher rejection rates and should be discussed with project maintainers before implementation, as also suggested in Pytorch contribution guide²⁵. Prior research [4] reports that challenging tasks can cause social fears for OSS newcomers, our study complements this by identifying the task types that are more suitable for ML OSS newcomers.

Peripheral contributors could persist in making contributions and gradually tackle more challenging tasks in order to progress toward Core contributors. Our finding in RQ1 highlights that factors like prolonged involvement, diverse file authorship, and collaboration are differentiating *Peripheral* contributors from *Core* contributors, which is consistent with the well-established OSS contributor categorization often described as the Onion model [74]. Moreover, our findings in RQ2 reveal that *Core* contributors tend to exhibit a progression toward higher technical importance within the first 6 months of activities in a project, whereas *Peripheral* contributors typically maintain the same level of technical importance upon joining the project. We conduct a trial experiment to analyze the trend of how *Core* contributors gradually work on various tasks. We randomly sample 20 *Core-Nighttime* contributors and analyze their commits before they make their most important commit (i.e., highest commit centrality) by dividing these commits into halves ordered by time. We observe that the commits of adding new features increase from 12.5% to 22.5%, indicating that these contributors gradually became more competent in implementing new features (e.g., adding an isnan operator²⁶). We also observe that they increasingly make commits related to environment-dependent processing

²⁵https://pytorch.org/docs/stable/community/contribution_guide.html#contribution-process

²⁶<https://github.com/apache/mxnet/commit/c92f95490e288e0658b4a349d48f95f1cf2d389d>

(e.g., optimizing GPU kernel implementations²⁷), from 12.5% to 17.5%, indicating that these contributors gradually started to handle either low-level hardware related code change or fix more complex software compatibility issues. Later, findings from RQ4 show that core contributors tend to shift away from the intensive coding-centric tasks and move towards more balanced contributions. This observed trajectory of increasing technical difficulty then shifts their work nature, quantitatively illustrates the long-term contributors' evolution within the ML OSS domain and adds to the existing perspective of contributor categorization.

7 Implication

In this section, we reflect on our findings and discuss their implications for OSS contributors, ML project maintainers and software engineering researchers.

7.1 Implications for OSS Contributors

ML Core contributors are suggested to split their pull requests into smaller, manageable segments to improve contribution efficiency and project productivity. In RQ1, our findings reveal that *Core* contributors, regardless of their contribute time, often make large and complex code changes (i.e., have a higher code contribution density and more discussions received in their pull requests). However, they demonstrate a lower pull request approval ratio and pull request approval density than *Peripheral* contributors. While prior research by Yu et al. [102] find that the number of lines of code added in pull requests negatively affects acceptance rates and increases review latency in non-ML OSS projects, our findings indicate that this issue is also prevalent in ML OSS projects and is even more severe among *Core* contributors. Therefore, we reinforce the need for ML contributors to adopt the established OSS practice of breaking down large tasks into smaller, independent pull requests to reduce review efforts and accelerate acceptance. Given the rapid pace of development in the ML field, this practice may help improve pull request approval rates and enhance the overall productivity of contributions to ML OSS projects.

7.2 Implications for project maintainers and managers

Project managers could prioritize allocating resources to issue and PR discussants and assign them more code reviews to enhance the project popularity. Our RQ3 findings show that the ratio of issue discussants in the project has a significant positive correlation with the project popularity, as users and contributors rely on discussants for problem-solving and feedback. To facilitate effective communications, project managers should train discussants to ensure timely and effective communication and allocate sufficient resources for issue handling, such as establishing clear guidelines for raising issues and considering hosting regular live Q&A sessions to address questions promptly and reduce the burden of asynchronous communication.

Project maintainers in established projects often document best practices for raising issues and participating in discussions in the contribution guide (e.g., Pytorch Ignite contribution guide²⁸) to enhance immediate productivity and workflow efficiency based on their daily experience. However, these guides often focus on short-term efficiency and may overlook practices that support long-term popularity growth. Identifying the factors that influence long-term popularity requires quantitative analysis of large-scale and long-term data, which is beyond the scope of short-term project management. Our findings based on analyzing historical data reveal a deeper connection between project popularity and the distribution of contributors with various workload compositions and their task assignments. To improve current contribution practices, project maintainers and managers could incorporate additional guidance that

²⁷<https://github.com/tensorflow/tensorflow/commit/a4bbf33e76e3de721a74230f7ea1f83e75f7c6ad>

²⁸<https://github.com/pytorch/ignite/blob/master/CONTRIBUTING.md>

promotes sustained popularity over time. For example, as highlighted in RQ3, project popularity growth is significantly positively correlated with the number of code reviews conducted by Issue Discussants and PR Discussants. Project maintainers can actively assign them to review code changes from newcomers, especially the newcomers who have previously interacted with in issue discussions. This strategy might potentially foster a welcoming environment for new contributors and improve their retention, as also shown in a real-world example in Figure 11 in Appendix D.

Leverage OSS engagement metrics to support progression and prevent drop-off. To prevent contributor drop-off and encourage progression towards more important roles, project managers can monitor contributors’ technical importance and contribution dynamics. For example, when a contributor shows increasing technical importance or maintains consistent contributions over a period of time, project managers can acknowledge their efforts through rewards such as issuing digital badges or public recognition. Tracking workload composition also enables maintainers to identify contributors who consistently focus on a single type of task and may be ready for new challenges. For instance, if an Issue Discussant frequently reports and discusses issues, project managers can invite them to begin contributing code, which encourages both engagement and skill growth.

7.3 Implications for software engineering researchers

Develop tools to assist in promoting project popularity. In RQ3, we find that project popularity (as measured by star and fork increases) is significantly correlated with the distribution of different types of contributors and the tasks assigned to them. While we acknowledge the limitations of stars and forks, such as the potential for manipulation (i.e. purchased stars [26, 42]), our observed correlations point to a promising direction for exploring how contributor task assignment may influence project popularity. Researchers could create monitoring systems to assess the health of contributor demographics and alert maintainers when imbalances occur between the project’s incoming workload and the contributors’ capacity. For example, in PyTorch Issue #107224²⁹, we observe that one contributor wishes to implement optimizers, while a follow-up comment says “*we don’t have any dedicated maintainers with sufficient bandwidth to help review and maintain C++ optimize changes at this time.*” This highlights how a lack of active reviewers for specific components can block incoming contributions. A monitoring tool could track the rising issue or pull request activities in specific components and detect stagnation among qualified reviewers—prompting maintainers to temporarily assign proxies or rearrange reviewers’ tasks to avoid discouraging contributors. In parallel, developing a detection tool that calculates contributors’ technical importance can help managers to identify the contributors who frequently work on central files in a given ML component. These contributors could then be invited to temporarily serve as proxy reviewers to ensure continued responsiveness.

Future research could also incorporate more popularity metrics, such as usage or download statistics, to enable a deeper understanding of popularity growth in OSS projects, especially when projects have a clear mapping between their GitHub repository and package distributions (e.g., PyPI). Additionally, tools could be developed to optimize task assignment strategies to sustain popularity growth. For instance, our finding that increased code reviews by Issue Discussants and PR Discussants are correlated with project popularity could be utilized to enhance code reviewer recommendation systems. The recommendation system could account for not only reviewers’ expertise and workload but also selecting reviewers who can welcome and retain newcomers.

Explore contributor burnout and role transition in the ML OSS community. Our findings in RQ4 reveal that long-term ML contributors tend to decrease their contributions over time and shift away from intensive technical tasks. This shift is particularly concerning given the diverse contribution types and specialized knowledge required in

²⁹<https://github.com/pytorch/pytorch/issues/107224>

ML projects, as well as the rapid evolutions in the ML field. Future research could explore the factors of contributor burnout within the ML OSS community and develop strategies to mitigate them. Furthermore, researchers can also examine burnout patterns among contributors working on different ML components, and develop approaches to help project maintainers anticipate and address the potential loss of critical specialized knowledge. In addition, to understand why contributors shift roles or what challenges contributors face when changing roles, we explore relevant discussion on GitHub. The PyTorch Issue #107224³⁰ as also mentioned above, shows that when project maintainers are scarce, even well-scoped contributors encounter delay. This reveals a challenge in role transition: the imbalance between development efforts and the accompanying review and maintenance workload. In fact, identifying the full range of rationales and challenges developers encounter when attempting to take on greater responsibilities is not straightforward. To fully understand contributors' motivations for changing their roles, future research is encouraged to conduct qualitative interviews with both active contributors and maintainers.

Explore employees versus volunteers in the ML OSS community. It is common that open-source ML libraries are supported by technology companies. Such companies play an important role in open-source governance and make concrete code contributions through their employees. Our profiles categorize ML contributors into *Daytime* versus *Nighttime* contribution patterns, enabling us to identify and quantify their differences empirically across various activities (e.g., pull requests, issues). Future research could explore how employees and volunteers contribute differently and the best practices for their collaborations. Moreover, finer-granularity studies can be conducted, such as how contributors from upstream vendors (e.g., Nvidia, AMD, Intel) and downstream organizations work together in the ML libraries. This insight could reveal strategies to enhance collaboration and productivity across both corporate-sponsored and community-driven contributions. In addition, open source ML projects can undergo particular OSS governance structures. For example, the Linux Foundation AI & Data Foundation serves as a hub where AI and data experts can collaborate together. It supports project lifecycles from sandbox, incubating, to graduated³¹. Future work can explore how contributors shift roles with respect to such governance structures and project stages, and what challenges contributor encounter when shifting roles.

8 Threats To Validity

In this section, we discuss the possible threats to the validity of our study.

Threats to Internal Validity concern the effectiveness of our analysis approaches. One internal threat to our results could be the effectiveness of the clustering techniques on grouping contributors. In this study, we employ various clustering techniques. We select the clustering result from eight clustering algorithms to identify the contributor profiles in Section 3. We also use hierarchical clustering to identify workload composition patterns in Section 4.1. To select the optimal number of clusters, we experiment with a range of number of clusters, and use the clustering evaluation metric (i.e., Silhouette score) to find the optimal one. For algorithms not requiring a predetermined cluster number, we conduct a gradient search to find the optimal parameter set that yields the highest Silhouette score. Altering the number of clusters may impact our results. To address this issue, for all the clustering results, beyond the clustering evaluation metric, we also manually investigate the datapoints in the resulting clusters to ensure the clustering yields meaningful grouping and insightful outcomes based on our designated features.

Our analysis distinguishes between Daytime and Nighttime contributors by assuming that developers typically follow a consistent contribution schedule. While this assumption captures the dominant behaviors for many contributors

³⁰<https://github.com/pytorch/pytorch/issues/107224>

³¹<https://lfaidata.foundation/blog/2023/06/07/collaborate-and-innovate-how-the-lf-ai-data-foundation-supports-open-source-projects/>

(using the mode of their contribution times as a proxy for their primary contribution period), it may not fully capture the behaviors of those with irregular schedules or those traveling across time zones. We acknowledge this limitation and encourage future research to explore these scenarios and better understand the impacts of irregular contributions.

Another potential internal threat is the effectiveness of the logistic regression model for modeling contributor profiles. We use binary logistic regression in RQ1 to identify important features for each profile, because it is a simple model that outputs the importance and direction of the associations between contributor features and profiles, which aligns with the objectives of RQ1. However, other ML algorithms could be applied in the future to further explore contributor characteristics.

The relationships identified in RQ3 are correlational rather than causal. Although our mixed-effect models identify significant correlations between project-popularity growth and contributors' workload compositions and work-preference features, the analysis is based on observable data from Github, while unmeasured factors may influence both popularity and contributor behaviour. We mitigate this threat by incorporating project-level random effects (i.e., project) and adding control covariates that capture known drivers of popularity (i.e., period). Such statistical modelling is a common practice in large-scale empirical software engineering research and offers a sound basis for correlation analysis; nevertheless, future work should conduct experiments to examine causality when validating or extending our results.

Threats to External Validity concern the generalizability of our results. Our study has been conducted over 8 ML libraries open source projects on Github. We also limit our study to analyzing ML library projects, instead of studying all the projects on Github that are implementing ML technology or developing ML libraries. To augment the generalizability of our result, we select the ML libraries open-source projects created for different purposes by different organizations with varying sizes to conduct our experiment. Additionally, our selected projects are mainly written in Python, C++, and Java. Including more ML libraries that are developed in other programming languages, such as Matlab, might reveal more meaningful findings regarding ML contributors. However, considering that Python is the most preferable programming language for scientific computing and machine learning [84], the generalizability of our results to other general ML projects is still adequate. On the other hand, our focus on the ML libraries and rather than general ML projects is motivated by the goal of studying contributors who possess deeper expertise in ML technologies. As such, our ML-specific findings may not generalize to all ML developers, particularly the developers who use ML tools in domain-specific applications without a focus on developing ML libraries. Nonetheless, our suggestions related to general OSS contributor engagement and progression remain applicable beyond ML-specific contexts. Moreover, the profiling methodology and analytical approaches we propose are generic and can be applied to general OSS projects beyond the ML domain in future research.

Threats to Construct Validity concern whether our analyzes measure what we claim to analyze. In our work, we aim to identify the profiles and understand the characteristics of ML OSS contributors. Although our dataset focuses on ML projects, there might still be traditional software development contributions irrelevant to ML knowledge. Unfortunately, we are not able to consult all contributors and ask for their experience in developing ML software. To minimize this threat, we select the ML projects that are developing ML frameworks and libraries instead of adopting ML techniques for applications in different domains. We believe that there is a greater portion of ML experts in projects developing ML libraries than in projects adopting ML tools.

In addition, during the calculation of the feature *number of issues solved*, we employ a keyword-based approach similar to prior studies [71, 99] to identify the issue-fixing commits or pull requests, and count the number of issues being solved accordingly. However, there might be cases that an issue is solved without making commits or pull requests,

or without mentioning the keywords. Future research is encouraged to consider such cases to further enhance the issue data quality.

In RQ3, we use the growth of project stars and forks as a measure of popularity, which poses a potential threat to construct validity due to the fact that GitHub stars can be artificially inflated or purchased [26]. However, the ML projects in our study are well-established and widely recognized in the AI community, which makes such manipulation relatively less likely. In addition, He et al. [42] report that fake stars are mostly used to promote short-lived malware repositories and the promotion tend to have only short-term effects. Therefore, we believe the impact of fake stars on our analysis is limited.

The last point is that this study examines the trend of how ML contributors contribute to open source ML libraries by analyzing the artifacts mined from GitHub repositories. However, external factors may affect contributors' contribution, such as new trends or breakthroughs in ML that could lead to new opportunities in ML library development. These external factors might influence how contributors engage in ML libraries. For example, Large Language Model (LLM) libraries, such as HuggingFace, have become increasingly active recently. Future research could explore how contributor engagement evolves in response to such technological trends within the ML ecosystem.

9 Related Work

9.1 Mono-Perspective OSS Contributor Categorization

Historical data in open-source software repositories can provide insights into software developers. Many existing studies in software engineering have categorized the behaviors of OSS contributors and evaluated their expertise in specific areas.

9.1.1 Core vs Peripheral Contributors

Researchers begin with classifying OSS contributors according to their amount of code contribution, typically into core, peripheral, and maybe other finer subgroups. Mockus et al. [64, 65] find that the top 20% active developers are usually the core developers who contribute more than 80% commits in OSS projects. Nakakoji et al. [74] propose an onion model to classify OSS community members into seven categories: project leaders, core members, active developers, peripheral developers, bug fixers, bug reporters, readers, and passive users. Milewicz et al. [62] find that open-source scientific software projects are driven by senior members of the team (e.g., professors), and they mainly tackle architectural problems, while junior members (e.g., graduate students) mainly work on implementing new features. Costa et al. [22] categorize developers as core, active, and peripheral, and find that core developers make the majority of contributions and are associated with less buggy commits.

In contrast to the prior studies that employed various approaches to specifically distinguish between *Core* and *Peripheral* developers, such as utilizing social-technical networks [52], our work focuses on understanding the contributor profiles in ML OSS projects by clustering the behaviors of contributors. Similar to the findings of prior research [22, 62, 97], our result shows that *Core* contributors make more coding contributions with large-scope file authorships, and their changes tend to be more technical and influential. *Peripheral* contributors tend to have a specialized and narrow focus in their coding efforts. Agreeing with [52, 62, 97], we also find *Core* contributors more involved in collaborative activities such as project management, mentoring, and code review than *Peripheral* contributors. Different from prior research, we further identify different behaviors within the *Core* contributor profile and *Peripheral* contributor profile in terms of day-time contributors and night-time contributors. We find *Core-Daytime* contributors work more on collaborative activities, while *Core-Daytime* contributors focus more on making complex code contributions. Additionally, our findings

align with Wang et al. [97] on observing that *Core* contributors tend to shift their focus to non-technical tasks (e.g., issue discussions) in their later stages. However, while they suggest that this shift negatively impacts project productivity and quality, we find that increases in issue discussants correlate with greater project popularity growth.

9.1.2 Contributor’s Duration, Technical Expertise, and Human Aspects

The duration of involvement in OSS projects is a key factor in categorizing contributors and has been extensively studied. Pinto et al. [83] find that half of the newcomers in a project only make one contribution and then depart from the project. Steinmacher et al. [89] find that more than 80% newcomers do not become long-term contributors. Zhou et al. [109], Wang et al. [96] and Bao et al. [7] develop prediction models to identify potential long-term contributors based on their initial behaviors in the project repository. In complementary of these studies, we identify the post-onboarding evolution of long-term contributors in ML projects, and find that they often experience technical progression within their first six active months and then shift away from the intensive technical contributions.

Technical expertise is an important aspect of differentiating contributors and identifying experts. Dey et al. [24] propose an approach to represent contributors’ expertise with their API usage. Dakhel et al. [23] represent the domain expertise of contributors based on the projects they have contributed to, issues solved, and their API usage. Ahasanuzzaman et al. [3] propose an approach that uses language-specific functions and APIs to assess one’s efficiency in a programming language. Montandon et al. [67] explore the effectiveness of clustering and machine learning classifiers in assessing contributors’ expertise and identifying experts based on GitHub activities. Yang et al. [101] develop a tool to visualize developers’ portraits based on their Github profile pages and coding details. In contrast to these specific technical expertise, we measure the technical importance of the contributions to the project codebase and track its evolutions. Additionally, we incorporate the number of programming languages as a technical expertise indicator for a contributor when creating profiles.

There are studies exploring the impact of human aspects on OSS contributors. Cataldo et al. [12] investigate the correlation between the quality of software projects and the geographical distributions of developers and find that projects with imbalanced distributions of the development team tend to have more defects compared to projects with balanced distributions. Claes et al. [16] conduct a large-scale study on the working hours of developers and find that approximately two-thirds of developers follow normal office hours, and no correlation between project maturation and the reduction in abnormal working hours. Yue et al. [103] find that the preference of contribution dynamics in the early career affects the contributor’s technical success in the project. In contrast, we study the work preferences of the contributor’s entire career in ML projects as well as the correlation of contributors’ work preferences with the project’s popularity. Elbaum et al. [70] identify a strong association between code churn and defects. Shin et al. [87] report code churn as one of the software vulnerability indicators. We use one human aspect - contribute time - to build contributor profiles, and model relationships with other human aspects (e.g., geographical location, Github followers). Notably, our analysis identifies the correlation between ML contributor’s geographic regions and contribution time.

Prior research primarily examines traditional software projects, while our study concentrates on contributors within popular machine learning projects. Unlike existing studies that often investigate contributors from individual angles (e.g., focusing exclusively on volume of code contribution, duration, geographical distributions, or technical expertise), we construct contributor profiles by considering multiple aspects, including their contribution volume, contribution time, contribution size, and technical expertise. Based on our comprehensive analysis, we report ML contributors’ multi-facet behaviors from both a static snapshot (e.g., distribution of workload compositions, contribution dynamics, technical importance, etc.) and a longitudinal perspective (e.g., long-term behavior evolutions). We also identify correlations

among behavioral features and connect contributor roles to project-level outcomes, such as popularity growth and role sustainability patterns within a project. These findings provide a comprehensive understanding of the characteristics and behaviors of members of the ML OSS community.

9.2 Research for Machine Learning Contributors

Research efforts to study ML software developers have been arising within the past few years. Cai et al. [11] deploy a survey to investigate the motivations, challenges, and demands of software developers when they begin learning ML, and find that the major challenge of ML beginners is the lack of understanding of ML concepts and mathematics, and they demand ML frameworks to provide such support. Hill et al. [43] interview 11 ML professionals from a company and find that their main challenge is creating a repeatable workflow for ML system development process. Ishikawa et al. [46] conduct a questionnaire survey on 278 ML practitioners and report the main difficulties perceived by ML contributors including low accuracy, absence of an oracle, and uncertainty regarding system behavior. Gangash et al. [6] conduct an empirical study analyzing ML-related posts on Stack Overflow and explore the topics frequently discussed by ML developers. Islam et al. [48] map the posts related to 10 popular ML libraries on Stack Overflow to ML system development stages. They identify model construction and data preparation as the most challenging stage of ML system development perceived by ML developers. Han et al. [38] study the onboard process for deep learning newcomers based on their activities before their first accepted commit in deep learning projects.

Previous research on ML contributors has primarily explored their motivations for learning ML and the challenges encountered in ML software development. These studies often use qualitative methods, such as surveys, interviews, and analyzing posts from Q&A systems to study ML developer's perceptions and self-reported barriers. In contrast, our approach leverages mining software repositories and conducting quantitative analysis to uncover behavioral patterns throughout contributors' diverse involvement in ML OSS projects. Our findings provide insights for addressing some practical challenges faced by ML developers, such as identifying beginner-friendly ML components and software engineering tasks to contribute to in ML OSS projects, identifying more complex ML components and SE tasks for contributors that aim to progress, and determining strategies to improve contribution acceptance.

10 Conclusion

This paper conducts an empirical study aimed at advancing the understanding of machine learning contributors within the machine learning open-source software community. By studying 11,949 contributors from 8 popular ML libraries, we identify four contributor profiles and reveal the most influential features associated with the profiles, such as project experience, authorship diversity, collaborations, and geographical location. We study the OSS engagement of contributors from four aspects: workload composition, work preferences, technical importance, and ML-specific contributions, and reveal the variations in the OSS engagement across contributors in different profiles. Moreover, we explore the correlation of contributor OSS engagement with the project popularity and identify the important factors of workload composition and work preference associated with the increase in project popularity. The study of the evolution of contributor's OSS engagement highlights that long-term contributors evolve towards fewer, constant, and balanced contribution dynamics, moving away from highly technical and intensive contributions. These findings contribute to a deeper understanding of the ML OSS ecosystem and offer practical implications for project maintainers and software engineering researchers aiming to foster a welcoming, efficient, and collaborative OSS environment. Our findings also offer suggestions for OSS contributors, including newcomers (i.e., potential contributors who have not yet

onboard), *Peripheral* contributors, and *Core* contributors to progress within ML projects. Moving forward, we aim to expand our empirical studies to include contributors from more ML frameworks and applications.

11 Replication Package

To facilitate the replication of our study, we provide our data and code publicly available at: https://github.com/Software-Evolution-Analytics-Lab-SEAL/ML_Contributor_replication.

References

- [1] [n. d.]. GitHub REST API. <https://docs.github.com/en/rest>. Accessed: 2024-04-09.
- [2] Ahmad Abdellatif, Mairieli Wessel, Igor Steinmacher, Marco A. Gerosa, and Emad Shihab. 2022. BotHunter: an approach to detect software bots in GitHub (MSR '22). Association for Computing Machinery, New York, NY, USA, 6–17. <https://doi.org/10.1145/3524842.3527959>
- [3] Md Ahasanuzzaman, Gustavo A Oliva, and Ahmed E Hassan. 2024. Using knowledge units of programming languages to recommend reviewers for pull requests: an empirical study. *Empirical Software Engineering* 29, 1 (2024), 33.
- [4] Sogol Balali, Umayal Annamalai, Hema Susmita Padala, Bianca Trinkenreich, Marco A. Gerosa, Igor Steinmacher, and Anita Sarma. 2020. Recommending Tasks to Newcomers in OSS Projects: How Do Mentors Handle It? (*OpenSym '20*). Association for Computing Machinery, New York, NY, USA, Article 7, 14 pages. <https://doi.org/10.1145/3412569.3412571>
- [5] Sogol Balali, Igor Steinmacher, Umayal Annamalai, Anita Sarma, and Marco Aurelio Gerosa. 2018. Newcomers' barriers... is that all? an analysis of mentors' and newcomers' barriers in OSS projects. *Computer Supported Cooperative Work (CSCW)* 27 (2018), 679–714.
- [6] Abdul Ali Bangash, Hareem Sahar, Shaiful Chowdhury, Alexander William Wong, Abram Hindle, and Karim Ali. 2019. What do Developers Know About Machine Learning: A Study of ML Discussions on StackOverflow. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. 260–264. <https://doi.org/10.1109/MSR.2019.00052>
- [7] Lingfeng Bao, Xin Xia, David Lo, and Gail C. Murphy. 2021. A large scale study of long-time contributor prediction for GitHub projects. *IEEE Transactions on Software Engineering* 47, 6 (1 June 2021), 1277–1298. <https://doi.org/10.1109/TSE.2019.2918536>
- [8] Marcus Vinicius Bertoncello, Gustavo Pinto, Igor Scalante Wiese, and Igor Steinmacher. 2020. Pull Requests or Commits? Which Method Should We Use to Study Contributors' Behavior?. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 592–601. <https://doi.org/10.1109/SANER48275.2020.9054855>
- [9] Viv Bewick, Liz Cheek, and Jonathan Ball. 2005. Statistics review 14: Logistic regression. *Critical care* 9, 1 (2005), 1–7.
- [10] Housseem Ben Braiek, Foutse Khomh, and Bram Adams. 2018. The open-closed principle of modern machine learning frameworks. In *Proceedings of the 15th International Conference on Mining Software Repositories*. 353–363.
- [11] Carrie J. Cai and Philip J. Guo. 2019. Software Developers Learning Machine Learning: Motivations, Hurdles, and Desires. In *2019 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 25–34. <https://doi.org/10.1109/VLHCC.2019.8818751>
- [12] Marcelo Cataldo and Sangeeth Nambiar. 2012. The impact of geographic distribution and the nature of technical coupling on the quality of global software development projects. *Journal of Software: Evolution and Process* 24 (03 2012), 153–168. <https://doi.org/10.1002/smr.477>
- [13] Junjie Chen, Yihua Liang, Qingchao Shen, Jiajun Jiang, and Shuochuan Li. 2023. Toward understanding deep learning framework bugs. *ACM Transactions on Software Engineering and Methodology* 32, 6 (2023), 1–31.
- [14] Davide Chicco and Giuseppe Jurman. 2020. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC genomics* 21 (2020), 1–13.
- [15] Maximilian Christ, Andreas W Kempa-Liehr, and Michael Feindt. 2016. Distributed and parallel time series feature extraction for industrial big data applications. *arXiv preprint arXiv:1610.07717* (2016).
- [16] Maëlick Claes, Mika V Mäntylä, Miikka Kuutila, and Bram Adams. 2018. Do programmers work at night or during the weekend?. In *Proceedings of the 40th International Conference on Software Engineering*. 705–715.
- [17] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin* 114, 3 (1993), 494.
- [18] Eldan Cohen and Mariano P Consens. 2018. Large-scale analysis of the co-commit patterns of the active developers in github's top repositories. In *2018 IEEE/ACM 15th International Conference on Mining Software Repositories (MSR)*. IEEE, 426–436.
- [19] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.
- [20] Jacob Cohen. 2013. *Statistical power analysis for the behavioral sciences*. Academic press.
- [21] David Roxbee Cox and Alan Stuart. 1955. Some quick sign tests for trend in location and dispersion. *Biometrika* 42, 1/2 (1955), 80–95.
- [22] Daniel Alencar da Costa, Uirá Kulesza, Eduardo Aranha, and Roberta Coelho. 2014. Unveiling developers contributions behind code commits: an exploratory study. In *Proceedings of the 29th Annual ACM Symposium on Applied Computing*. 1152–1157.
- [23] Arghavan Moradi Dakhel, Michel C. Desmarais, and Foutse Khomh. 2023. Dev2vec: Representing domain expertise of developers in an embedding space. *Information and Software Technology* 159 (July 2023), 107218. <https://doi.org/10.1016/j.infsof.2023.107218>
- [24] Tapajit Dey, Andrey Karnauch, and Audris Mockus. 2021. Representation of developer expertise in open source software. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 995–1007.

- [25] Malinda Dilhara, Ameya Ketkar, and Danny Dig. 2021. Understanding software-2.0: A study of machine learning library usage and evolution. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 4 (2021), 1–42.
- [26] Kun Du, Hao Yang, Yubao Zhang, Haixin Duan, Haining Wang, Shuang Hao, Zhou Li, and Min Yang. 2020. Understanding promotion-as-a-service on GitHub. In *Proceedings of the 36th Annual Computer Security Applications Conference*. 597–610.
- [27] Mariam El Mezouar, Feng Zhang, and Ying Zou. 2019. An empirical study on the teams structures in social coding using GitHub projects. *Empirical Software Engineering* 24, 6 (2019), 3790–3823.
- [28] Patrick Loic Foale, Foutse Khomh, and Heng Li. 2024. Studying logging practice in machine learning-based applications. *Information and Software Technology* 170 (2024), 107450.
- [29] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. 2010. Regularization Paths for Generalized Linear Models via Coordinate Descent. , 22 pages. <https://www.jstatsoft.org/v33/i01/>
- [30] Jiri Gesi, Siqi Liu, Jiawei Li, Iftekhar Ahmed, Nachiappan Nagappan, David Lo, Eduardo Santana de Almeida, Pavneet Singh Kochhar, and Lingfeng Bao. 2022. Code smells in machine learning systems. *arXiv preprint arXiv:2203.00803* (2022).
- [31] Amin Ghadesi, Maxime Lamothe, and Heng Li. 2023. What Causes Exceptions in Machine Learning Applications? Mining Machine Learning-Related Stack Traces on Stack Overflow. *arXiv preprint arXiv:2304.12857* (2023).
- [32] Danielle Gonzalez, Thomas Zimmermann, and Nachiappan Nagappan. 2020. The state of the ml-universe: 10 years of artificial intelligence & machine learning software development on github. In *Proceedings of the 17th International conference on mining software repositories*. 431–442.
- [33] Hao Guan, Ying Xiao, Jiaying Li, Yepang Liu, and Guangdong Bai. 2023. A Comprehensive Study of Real-World Bugs in Machine Learning Model Optimization. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 147–158. <https://doi.org/10.1109/ICSE48619.2023.00024>
- [34] Philip Guo. 2018. How did people write machine learning code in the past? (2018). <https://cacm.acm.org/blogs/blog-cacm/230805-how-did-people-write-machine-learning-code-in-the-past/fulltext>
- [35] Junxiao Han, Shuiguang Deng, David Lo, Chen Zhi, Jianwei Yin, and Xin Xia. 2020. An empirical study of the dependency networks of deep learning libraries. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 868–878.
- [36] Junxiao Han, Jiakun Liu, David Lo, Chen Zhi, Yishan Chen, and Shuiguang Deng. 2023. On the sustainability of deep learning projects: Maintainers’ perspective. *Journal of Software: Evolution and Process* (2023), e2645.
- [37] Junxiao Han, Emad Shihab, Zhiyuan Wan, Shuiguang Deng, and Xin Xia. 2020. What do programmers discuss about deep learning frameworks. *Empirical Software Engineering* 25 (2020), 2694–2747.
- [38] Junxiao Han, Jiahao Zhang, David Lo, Xin Xia, Shuiguang Deng, and Minghui Wu. 2024. Understanding Newcomers’ Onboarding Process in Deep Learning Projects. *IEEE Transactions on Software Engineering* (2024), 1–18. <https://doi.org/10.1109/TSE.2024.3353297>
- [39] Nima Shiri Harzevili, Jiho Shin, Junjie Wang, Song Wang, and Nachiappan Nagappan. 2023. Automatic Static Vulnerability Detection for Machine Learning Libraries: Are We There Yet?. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 795–806.
- [40] Nima Shiri Harzevili, Jiho Shin, Junjie Wang, Song Wang, and Nachiappan Nagappan. 2023. Characterizing and understanding software security vulnerabilities in machine learning libraries. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 27–38.
- [41] Safwat Hassan, Chakkrit Tantithamthavorn, Cor-Paul Bezemer, and Ahmed E Hassan. 2018. Studying the dialogue between users and developers of free apps in the google play store. *Empirical Software Engineering* 23 (2018), 1275–1312.
- [42] Hao He, Haoqin Yang, Philipp Burckhardt, Alexandros Kapravelos, Bogdan Vasilescu, and Christian Kästner. 2024. 4.5 Million (Suspected) Fake Stars in GitHub: A Growing Spiral of Popularity Contests, Scams, and Malware. *arXiv preprint arXiv:2412.13459* (2024).
- [43] Charles Hill, Rachel Bellamy, Thomas Erickson, and Margaret Burnett. 2016. Trials and tribulations of developers of intelligent systems: A field study. In *2016 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 162–170. <https://doi.org/10.1109/VLHCC.2016.7739680>
- [44] Jin Huang and C.X. Ling. 2005. Using AUC and accuracy in evaluating learning algorithms. *IEEE Transactions on Knowledge and Data Engineering* 17, 3 (2005), 299–310. <https://doi.org/10.1109/TKDE.2005.50>
- [45] Nargiz Humbatova, Gunel Jahangirova, Gabriele Bavota, Vincenzo Riccio, Andrea Stocco, and Paolo Tonella. 2020. Taxonomy of real faults in deep learning systems. In *Proceedings of the ACM/IEEE 42nd international conference on software engineering*. 1110–1121.
- [46] Fuyuki Ishikawa and Nobukazu Yoshioka. 2019. How Do Engineers Perceive Difficulties in Engineering of Machine-Learning Systems? - Questionnaire Survey. In *2019 IEEE/ACM Joint 7th International Workshop on Conducting Empirical Studies in Industry (CESI) and 6th International Workshop on Software Engineering Research and Industrial Practice (SER&IP)*. 2–9. <https://doi.org/10.1109/CESER-IP.2019.00009>
- [47] Md Johirul Islam, Giang Nguyen, Rangeet Pan, and Hridesh Rajan. 2019. A comprehensive study on deep learning bug characteristics. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 510–520.
- [48] Md Johirul Islam, Hoan Anh Nguyen, Rangeet Pan, and Hridesh Rajan. 2019. What do developers ask about ml libraries? a large-scale study using stack overflow. *arXiv preprint arXiv:1906.11940* (2019).
- [49] Md Johirul Islam, Rangeet Pan, Giang Nguyen, and Hridesh Rajan. 2020. Repairing deep neural networks: Fix patterns and challenges. In *Proceedings of the ACM/IEEE 42nd international conference on software engineering*. 1135–1146.
- [50] Hadhemi Jebnoun, Housseem Ben Braiek, Mohammad Masudur Rahman, and Foutse Khomh. 2020. The scent of deep learning code: An empirical study. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 420–430.
- [51] Xiaoyang Jiang, Xingzhou Zhang, and Xiaohui Peng. 2023. Flet-Edge: A Full Life-cycle Evaluation Tool for deep learning framework on the Edge. In *2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 680–687.

- [52] Mitchell Joblin, Sven Apel, Claus Hunsen, and Wolfgang Mauerer. 2017. Classifying developers into core and peripheral: An empirical study on count and network metrics. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 164–174.
- [53] Tulasi Kavarakuntla, Liangxiu Han, Huw Lloyd, Annabel Latham, Anthony Kleerekoper, and Samson B Akintoye. 2024. A Generic Performance Model for deep learning in a Distributed Environment. *IEEE Access* (2024).
- [54] Samantha Syeda Khairunnisa, Shabbir Ahmed, Sayem Mohammad Imtiaz, Hridesh Rajan, and Gary T Leavens. 2023. What kinds of contracts do ML APIs need? *Empirical Software Engineering* 28, 6 (2023), 142.
- [55] Zhongzheng Lai, Huaming Chen, Ruoxi Sun, Yu Zhang, Minhui Xue, and Dong Yuan. 2024. On Security Weaknesses and Vulnerabilities in Deep Learning Systems. *arXiv preprint arXiv:2406.08688* (2024).
- [56] Huashan Lei, Shuai Zhang, Jun Wang, Guanping Xiao, Yepang Liu, and Yulei Sui. 2023. Why Do Deep Learning Projects Differ in Compatible Framework Versions? An Exploratory Study. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 509–520.
- [57] Jenny T Liang, Thomas Zimmermann, and Denae Ford. 2022. Towards mining oss skills from github activity. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 106–110.
- [58] Chao Liu, Runfeng Cai, Yiqun Zhou, Xin Chen, Haibo Hu, and Meng Yan. 2024. Understanding the implementation issues when using deep learning frameworks. *Information and Software Technology* 166 (2024), 107367.
- [59] Guoming Long and Tao Chen. 2022. On reporting performance and accuracy bugs for deep learning frameworks: An exploratory study from github. In *Proceedings of the 26th International Conference on Evaluation and Assessment in Software Engineering*. 90–99.
- [60] Nikolaos Louloudakis, Perry Gibson, José Cano, and Ajitha Rajan. 2023. Fault Localization for Buggy Deep Learning Framework Conversions in Image Recognition. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1795–1799.
- [61] Jeremy Miles. 2005. R-squared, adjusted R-squared. *Encyclopedia of statistics in behavioral science* (2005).
- [62] Reed Milewicz, Gustavo Pinto, and Paige Rodeghero. 2019. Characterizing the Roles of Contributors in Open-Source Scientific Software Projects. <https://doi.org/10.1109/MSR.2019.00069>
- [63] Ran Mo, Yao Zhang, Yushuo Wang, Siyuan Zhang, Pu Xiong, Zengyang Li, and Yang Zhao. 2023. Exploring the Impact of Code Clones on Deep Learning Software. *ACM Transactions on Software Engineering and Methodology* 32, 6 (2023), 1–34.
- [64] Audris Mockus, Roy T Fielding, and James Herbsleb. 2000. A case study of open source software development: the Apache server. In *Proceedings of the 22nd international conference on Software engineering*. 263–272.
- [65] Audris Mockus, Roy T Fielding, and James D Herbsleb. 2002. Two case studies of open source software development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 11, 3 (2002), 309–346.
- [66] Joao Eduardo Montandon, Luciana Lourdes Silva, and Marco Tulio Valente. 2019. Identifying experts in software libraries and frameworks among GitHub users. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 276–287.
- [67] João Eduardo Montandon, Luciana Lourdes Silva, and Marco Túlio Valente. 2019. Identifying Experts in Software Libraries and Frameworks among GitHub Users. *CoRR abs/1903.08113* (2019). [arXiv:1903.08113](http://arxiv.org/abs/1903.08113) <http://arxiv.org/abs/1903.08113>
- [68] David S Moore. 1977. Generalized inverses, Wald’s method, and the construction of chi-squared tests of fit. *J. Amer. Statist. Assoc.* 72, 357 (1977), 131–137.
- [69] Mohammad Mehdi Morovati, Amin Nikanjam, Florian Tambon, Foutse Khomh, and Zhen Ming Jiang. 2024. Bug characterization in machine learning-based systems. *Empirical Software Engineering* 29, 1 (2024), 14.
- [70] John C Munson and Sebastian G Elbaum. 1998. Code churn: A measure for estimating the impact of code change. In *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*. IEEE, 24–31.
- [71] Biruk Asmare Muse, Mohammad Masudur Rahman, Csaba Nagy, Anthony Cleve, Foutse Khomh, and Giuliano Antoniol. 2020. On the prevalence, impact, and evolution of sql code smells in data-intensive systems. In *Proceedings of the 17th international conference on mining software repositories*. 327–338.
- [72] Nadim Nachar et al. 2008. The Mann-Whitney U: A test for assessing whether two independent samples come from the same distribution. *Tutorials in quantitative Methods for Psychology* 4, 1 (2008), 13–20.
- [73] Shinichi Nakagawa, H Schielzeth, and Robert B O’Hara. 2013. A general and simple method for obtaining R² from generalized linear mixed-effects models. *Methods Ecol Evol* 4: 133–142.
- [74] Kumiyo Nakakoji, Yasuhiro Yamamoto, Yoshiyuki Nishinaka, Kouichi Kishida, and Yunwen Ye. 2002. Evolution patterns of open-source software systems and communities. In *Proceedings of the international workshop on Principles of software evolution*. 76–85.
- [75] Arunkaleeshwaran Narayanan, Junjie Wang, Lin Shi, Moshi Wei, Song Wang, et al. 2023. Automatic Unit Test Generation for Deep Learning Frameworks based on API Knowledge. *arXiv preprint arXiv:2307.00404* (2023).
- [76] Ehsan Noei, Feng Zhang, and Ying Zou. 2021. Too Many User-Reviews! What Should App Developers Look at First? *IEEE Transactions on Software Engineering* 47, 2 (2021), 367–378. <https://doi.org/10.1109/TSE.2019.2893171>
- [77] S. Noei, H. Li, S. Georgiou, and Y. Zou. 2023. An Empirical Study of Refactoring Rhythms and Tactics in the Software Development Process. *IEEE Transactions on Software Engineering* 49, 12 (dec 2023), 5103–5119. <https://doi.org/10.1109/TSE.2023.3326775>
- [78] Shayan Noei, Heng Li, and Ying Zou. 2024. Detecting Refactoring Commits in Machine Learning Python Projects: A Machine Learning-Based Approach. *ACM Transactions on Software Engineering and Methodology* (2024).
- [79] Evangelia Panourgia, Theodoros Plessas, Ilias Balampanis, and Diomidis Spinellis. 2024. Good Tools are Half the Work: Tool Usage in Deep Learning Projects. *arXiv:2310.19124 [cs.SE]* <https://arxiv.org/abs/2310.19124>

- [80] Karl Pearson. 1900. X. On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 50, 302 (July 1900), 157–175. <https://doi.org/10.1080/14786440009463897>
- [81] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [82] José Pinheiro and Douglas Bates. 2006. *Mixed-effects models in S and S-PLUS*. Springer science & business media.
- [83] Gustavo Pinto, Igor Steinmacher, and Marco Aurélio Gerosa. 2016. More common than you think: An in-depth study of casual contributors. In *2016 IEEE 23rd international conference on software analysis, evolution, and reengineering (SANER)*, Vol. 1. IEEE, 112–123.
- [84] Sebastian Raschka, Joshua Patterson, and Corey Nolet. 2020. Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *Information* 11, 4 (2020), 193.
- [85] Skipper Seabold and Josef Perktold. 2010. statsmodels: Econometric and statistical modeling with python. In *9th Python in Science Conference*.
- [86] Jiho Shin, Moshi Wei, Junjie Wang, Lin Shi, and Song Wang. 2023. The good, the bad, and the missing: Neural code generation for machine learning tasks. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–24.
- [87] Yonghee Shin, Andrew Meneely, Laurie Williams, and Jason A Osborne. 2010. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. *IEEE transactions on software engineering* 37, 6 (2010), 772–787.
- [88] Davide Spadini, Mauricio Aniche, and Alberto Bacchelli. 2018. PyDriller: Python framework for mining software repositories. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Lake Buena Vista, FL, USA) (ESEC/FSE 2018)*. Association for Computing Machinery, New York, NY, USA, 908–911. <https://doi.org/10.1145/3236024.3264598>
- [89] Igor Steinmacher, Igor Wiese, Ana Paula Chaves, and Marco Aurélio Gerosa. 2013. Why do newcomers abandon open source software projects?. In *2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. IEEE, 25–32.
- [90] Xiaobing Sun, Tianchi Zhou, Gengjie Li, Jiajun Hu, Hui Yang, and Bin Li. 2017. An empirical study on real bugs for machine learning programs. In *2017 24th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 348–357.
- [91] Xin Tan, Kai Gao, Minghui Zhou, and Li Zhang. 2022. An exploratory study of deep learning supply chain. In *Proceedings of the 44th International Conference on Software Engineering*. 86–98.
- [92] Lionel Nganyewou Tidjon, Ben Rombaut, Foutse Khomh, Ahmed E Hassan, et al. 2022. An empirical study of library usage and dependency in deep learning frameworks. *arXiv preprint arXiv:2211.15733* (2022).
- [93] Hiro Y Toda and Taku Yamamoto. 1995. Statistical inference in vector autoregressions with possibly integrated processes. *Journal of econometrics* 66, 1-2 (1995), 225–250.
- [94] Jun Wang, Guanping Xiao, Shuai Zhang, Huashan Lei, Yepang Liu, and Yulei Sui. 2023. Compatibility issues in deep learning systems: Problems and opportunities. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 476–488.
- [95] Song Wang, Nishtha Shrestha, Abarna Kucheri Subburaman, Junjie Wang, Moshi Wei, and Nachiappan Nagappan. 2021. Automatic unit test generation for machine learning libraries: How far are we?. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1548–1560.
- [96] Tao Wang, Yang Zhang, Gang Yin, Yue Yu, and Huaimin Wang. 2018. Who will become a long-term contributor? A prediction model based on the early phase behaviors. In *Proceedings of the 10th Asia-Pacific Symposium on Internetware*. 1–10.
- [97] Zhendong Wang, Yang Feng, Yi Wang, James A Jones, and David Redmiles. 2020. Unveiling elite developers’ activities in open source projects. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 29, 3 (2020), 1–35.
- [98] Zhaobin Wang, Ke Liu, Jian Li, Ying Zhu, and Yaonan Zhang. 2019. Various frameworks and libraries of machine learning and deep learning: a survey. *Archives of computational methods in engineering* (2019), 1–24.
- [99] Fengcai Wen, Csaba Nagy, Michele Lanza, and Gabriele Bavota. 2022. Quick remedy commits and their impact on mining software repositories. *Empirical Software Engineering* 27 (2022), 1–31.
- [100] Chunqiu Steven Xia, Saikat Dutta, Sasa Misailovic, Darko Marinov, and Lingming Zhang. 2023. Balancing effectiveness and flakiness of non-deterministic machine learning tests. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1801–1813.
- [101] Wenhua Yang, Minxue Pan, Yu Zhou, and Zhiqiu Huang. 2020. Developer portraying: A quick approach to understanding developers on OSS platforms. *Information and Software Technology* 125 (2020), 106336. <https://doi.org/10.1016/j.infsof.2020.106336>
- [102] Yue Yu, Gang Yin, Tao Wang, Cheng Yang, and Huaimin Wang. 2016. Determinants of pull-based development in the context of continuous integration. *Science China Information Sciences* 59 (2016), 1–14.
- [103] Yang Yue, Yi Wang, and David Redmiles. 2022. Off to a Good Start: Dynamic Contribution Patterns and Technical Success in an OSS Newcomer’s Early Career. *IEEE Transactions on Software Engineering* 49, 2 (2022), 529–548.
- [104] Jerrold H Zar. 2005. Spearman rank correlation. *Encyclopedia of biostatistics* 7 (2005).
- [105] Tianyi Zhang, Cuiyun Gao, Lei Ma, Michael Lyu, and Miryung Kim. 2019. An empirical study of common challenges in developing deep learning applications. In *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 104–115.
- [106] Tianyi Zhang, Cuiyun Gao, Lei Ma, Michael Lyu, and Miryung Kim. 2019. An Empirical Study of Common Challenges in Developing Deep Learning Applications. In *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. 104–115. <https://doi.org/10.1109/ISSRE.2019>

00020

- [107] Yuxia Zhang, Mian Qin, Klaas-Jan Stol, Minghui Zhou, and Hui Liu. 2024. How Are Paid and Volunteer Open Source Developers Different? A Study of the Rust Project. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [108] Zhongkai Zhao, Bonan Kou, Mohamed Yilmaz Ibrahim, Muhao Chen, and Tianyi Zhang. 2023. Knowledge-based version incompatibility detection for deep learning. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 708–719.
- [109] Minghui Zhou and Audris Mockus. 2015. Who Will Stay in the FLOSS Community? Modeling Participant’s Initial Behavior. *IEEE Transactions on Software Engineering* 41, 1 (2015), 82–99. <https://doi.org/10.1109/TSE.2014.2349496>
- [110] Yiren Zhou, Yu Qiao, and Tao Xu. 2023. Exploring the Characteristics of Popular Deep Learning GitHub Repositories. In *2023 International Conference on Intelligent Computing and Next Generation Networks (ICNGN)*. IEEE, 1–6.

A Pytorch PR Merge Status

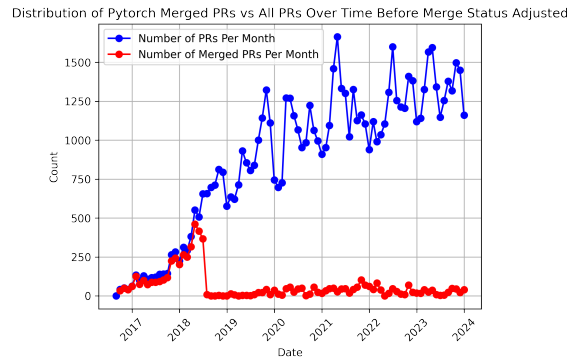


Fig. 8. The number of pull requests raised in PyTorch per month and the number of pull requests merged, only based on GitHub’s pull request merged status.

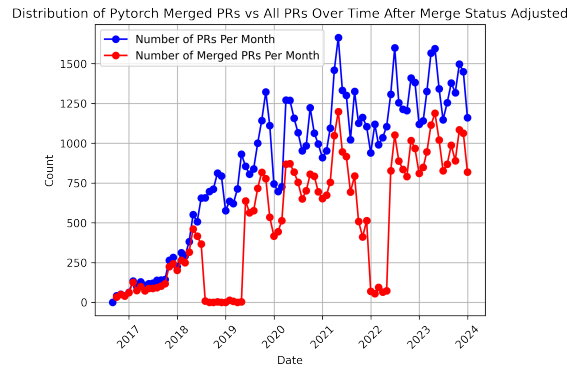


Fig. 9. The number of pull requests raised in PyTorch per month and the number of pull requests merged, based on both GitHub’s pull request merged status and the ‘Merged’ label.

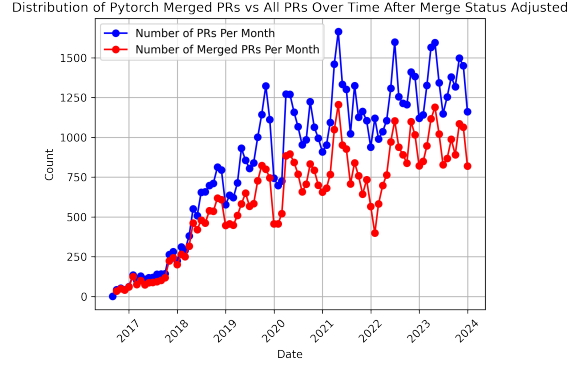


Fig. 10. The number of pull requests raised in PyTorch per month and the number of pull requests merged, based on the combination of three methods: GitHub’s pull request merged status, the ‘Merged’ label, and the presence of commit SHA in PR closing event.

As shown in Figure 8, prior to July 2018, Pytorch set accepted pull requests to ‘Merged’ status, but after July 2018, very few pull requests (i.e., less than 100 each month) were set to ‘Merged’ status. A comment in pull request³² mentions Pytorch uses facebook-github-bot to copy the approved commits into an internal repository, and another comment in pull request³³ mentions that facebook-github-bot automatically closes the PR on the Github repository after merging the changes into the internal repository. We find that most pull requests are closed by the bot, with the status always set to ‘Closed,’ regardless of whether the pull request was merged. However, contributors with merge permissions (e.g., pull request³⁴) could bypass the bot and set the pull request close status to ‘Merged’ instead of ‘Closed’.

As shown in Figure 9, PyTorch assigns a ‘Merged’ label to the approved and merged pull requests starting in May 2019. This begins with pull request³⁵ on October 6, 2020, the facebook-github-bot starts assigning a ‘Merged’ label when closing pull requests that are merged. Additionally, the ‘Merged’ label is retroactively applied to pull requests closed before this date, up until July 2019 (note: some older pull requests are labeled ‘merged’ with a lowercase, which no longer exists in the Pytorch label system and cannot be collected into our dataset). Therefore, we consider both the ‘Merged’ status and the ‘Merged’ label as the indication of acceptance for PyTorch pull requests before July 2018 and after May 2019.

However, between July 2018 and May 2019, and between December 2021 and May 2022, few pull requests were set to ‘Merged’ status or assigned a ‘Merged’ label, even though the pull request submission rate was similar to other periods (as shown by the two intervals with inconsistent PR acceptance in Figure 9). We manually investigate the comments under the pull requests submitted during this period, and find that many without a ‘Merged’ label or ‘Merged’ status are actually accepted. We could not find further documentation or official announcements from the PyTorch team about these two periods. To address these false negatives, we follow the approach proposed by Bertonecello et al. [8], which reports that some projects include the commit SHA in the merged PRs’ close event. We find that PyTorch also applies this practice. Therefore, we combine three methods to identify merged PRs in PyTorch: GitHub’s ‘Merged’ status, the ‘Merged’ label assigned by project maintainers, and PRs closed with a commit SHA. As shown in Figure 10, this approach results in a relatively consistent PR approval rate aligned with PR submission rates across all time periods.

³²<https://github.com/pytorch/pytorch/pull/8581>

³³<https://github.com/pytorch/pytorch/pull/8591>

³⁴<https://github.com/pytorch/pytorch/pull/135068>

³⁵<https://github.com/pytorch/pytorch/pull/45773>

B Contributor Features

Table 12. List of contributor features

Category	Feature	Description (D) & Rationale (R)	Reference
Personal Attributes	Timezone	D: The primary timezone of a developer (i.e., the most frequent timezone where a contributor makes commits). In this study, we define the timezone range of -13 to -2 as Americas, -1 to 3 as Europe/Africa, and 4 to 12 as Asia. R: Reveals the global distribution of ML contributors and how it affect collaboration patterns across different regions.	[12, 77]
	Contribute Time	D: The primary work time of a developer (i.e., the most frequent commit time of a contributor). Daytime are defined as 8h to 18h in this study. R: Reveals when contributors are most active and their working habits (i.e., whether a contributor would like to contribute to OSS in a work-like manner).	[16, 77]
	Duration	D: Number of days a developer stays in a project (number of days between the first and the last activity of a contributor). R: Shows a developer's experience with a project.	[24, 77]
	Number of followers	D: Number of Github followers a developer has. R: Reflects the contributor's social influence within the OSS community.	[7, 96, 101]
	Number of collaborated developers	D: Number of developers in the project a developer has collaborated with. Collaboration occurs when a developer interacts with another by commenting on their pull request or issue report, reviewing their pull request, or when the other developer does the same in return. R: Reflects the developer's collaborative skills and social influence in the project.	
	Number of authored files	D: The number of files in the project repository a developer has made changes to. R: Reflects the breadth and scope of the developer's authorship (i.e., whether focusing on specific areas or working on a wide range of areas).	[24, 83]
	Number of programming languages	D: The number of programming languages a developer has used in one's code contributions. R: Reflects the contributor's technical expertise across different components of ML frameworks. For example, proficiency in multiple languages, such as C++ for low-level operations and Python for high-level APIs shows the contributor's ability to work across multiple ML components.	[101]
	Number of commits	D: The number of commits submitted by a developer. R: Measures a contributor's cumulative effort towards modifying the project codebase.	[7, 24, 67], [52, 77, 96]
	Commit rate	D: Number of commits / Duration R: Reflects the intensity and frequency of a contributor's commit activity, as Montandonet al.[67] highlight that both the frequency and breadth of changes are valuable in assessing a developer's expertise.	[67, 107]
	Number of code change commits	D: The number of commits containing changes to source code files. R: Measures the developer's cumulative efforts to improve the project's functionalities (e.g., add features, and fix bugs).	

Continued on next page

Category	Feature	Description (D) & Rationale (R)	Reference
Commit	Code commit rate	D: The number of code change commits / Duration R: Measures the frequency of a contributor's efforts to improve the project's functionalities.	
	Number of non-code change commits	D: The number of commits made by a developer that do not include changes to source code files (e.g., updating documentation, tutorials, and build files). R: Measures the developer's cumulative non-coding efforts.	[57]
	Non-code change commit rate	D: The number of non-code commits / Duration R: Measures the frequency of a contributor's efforts to make non-code changes.	
	Code contribution	D: Total number of lines of code added or deleted by a developer. R: Provides a finer-grained measure of a developer's coding effort compared to commits, which capture the magnitude of code changes more precisely.	[52, 83, 101]
	Code contribution rate	D: Code contribution / Duration R: Measures the frequency of a developer's code changes at LOC level.	
	Code contribution density	D: Average code churn per commit. R: Reflects the magnitude of changes made by a contributor. Differentiate minor updates (e.g., small fixes or tweaks) and significant development efforts (e.g., adding new features or refactoring large portions of code).	[67, 77]
Issue	Number of issues	D: Number of issue reports raised by a developer. R: Measures a contributor's contribution in identifying problems, suggesting improvements, and proposing new features for the project.	[7, 38, 96]
	Number of issues participated	D: The number of issue reports raised by other developers that a developer has participated in. Examples of participating activities can be commenting, triggering events, triaging, or being assigned to resolve the issue. This information is obtained from the list of issue participants using the GitHub REST API. R: Reveals a developer's collaborative skills and community involvement through issue report engagement.	[96]
	Average issue comments received	D: The average number of comments received on a developer's issue reports. R: Reflects the importance or complexity of the issue reports raised by a developer. More important or complex issues typically attract more comments.	[7]
	Number of issues solved	D: Number of issue reports the developer has resolved. To get this value, we first parse the commit message of all commits and the title and description of pull requests made in the studied time period. If the commit message or pull request contains keywords such as "fix", "resolve", "address", "close" or "solve" together with a valid issue id in the same line, this commit is considered solving the issue and the author of this commit is considered the one who solves the issue. Note that the issue must be raised before the time of the commit is made. Then, we use the same method to find issue ids from pull request messages. This feature specifically captures issues resolved through commits and does not include those resolved through discussions in the issue report alone. R: Reveals a developer's code contribution to fixing bugs and addressing project problems through their commits.	[23, 71, 99]

Continued on next page

Category	Feature	Description (D) & Rationale (R)	Reference
	Issue contribution	D: Total number of issues raised or issues solved by a developer. R: Reflects a developer's collaborative efforts in identifying and resolving issues.	[89]
	Issue contribution rate	D: Issue Contribution / Duration R: Measures the frequency of a developer's contribution to identifying and resolving issues.	
	Issue solving ratio	D: Issue solved / Number of issues participated R: Reflects the efficiency of a contributor in solving the issues they participated.	
	Issue solving rate	D: Issue solved / Duration R: Measures the frequency of a developer's contribution to resolving issues.	
Pull Request	Number of pull requests	D: The number of pull requests submitted by a developer. R: Pull requests capture a contributor's efforts to change the codebase at a higher abstraction level than commits, as they often contain multiple commits for the same change and involve review, discussion, and collaboration. The total number of pull requests reflects the amount of reviewed changesets the developer has proposed to the project.	[7, 38, 96] [8]
	Number of PR participated	D: The number of pull requests submitted by other developers that a developer has participated in. Examples of participating activities can be commenting, triggering events, triaging, or reviewing. This information is obtained from the list of PR participants using the GitHub REST API. R: Reveals a developer's collaborative skills and their willingness to contribute to the quality of others' code contributions.	[57, 96]
	Number of PR reviewed	D: The number of pull requests reviewed by a developer. R: Measures a contributor's code review contributions.	[38]
	Number of PR merged	D: The number of pull requests submitted by a developer being merged. R: Measures the developer's accepted contributions to the codebase, which reflect the amount of high-quality and project-aligned changes.	[7]
	PR approval ratio	D: Number of PR merged / Number of pull requests R: Assesses the quality and alignment of a contributor's work with project standards.	
	Average PR comments received	D: The average number of comments received on a developer's pull requests. R: Reflects the importance or complexity of the code changes made by a developer. More important or complex code changes typically require more discussions and revisions before they can be merged.	[7]
	PR contribution	D: Number of pull requests submitted by a developer that has been merged and the number of pull requests reviewed by the developer. R: PR contribution reflects a developer's direct impact on the quality of the codebase, as merged pull requests introduce changes, and reviewing them ensures the changes meet the project's quality standards.	
	PR contribution rate	D: PR contribution / Duration R: Reflects how frequently developers contribute to or review code change.	

C RQ1 Supplementary Result

Table 13. Result of Mann-Whitney U tests and effect sizes for contributor features.

Contributor Features	Group1: Core Group2: Peripheral	Group1: Core-Nighttime Group2: Core-Daytime	Group1: Peripheral-Nighttime Group2: Peripheral-Daytime
Timezone	small (-0.238)	small (0.319)	small (0.203)
Contribute Time	not significant	small (-0.177)	small (-0.148)
Duration	large (0.555)	negligible (-0.143)	not significant
Followers	negligible (0.081)	not significant	not significant
Total Commits	large (0.769)	small (-0.188)	not significant
Collaborations	medium (0.466)	not significant	negligible (0.062)
Authored Files	large (0.924)	negligible (-0.109)	not significant
Languages	large (0.856)	not significant	negligible (0.023)
Commit Rate	negligible (-0.03)	negligible (-0.077)	not significant
Code Commits	large (0.787)	small (-0.182)	not significant
Code Commit Rate	small (0.195)	negligible (-0.078)	not significant
Non Code Commits	medium (0.453)	small (-0.164)	not significant
Non Code Commit Rate	small (0.262)	negligible (-0.126)	not significant
Code Contribution	large (0.963)	negligible (-0.111)	not significant
Code Contribution Rate	large (0.657)	not significant	not significant
Code Contribution Density	large (0.906)	negligible (0.069)	not significant
Total Issues	small (0.231)	negligible (0.061)	negligible (0.071)
Issue Participated	medium (0.444)	negligible (-0.056)	negligible (0.038)
Issue Comments Received	small (0.172)	negligible (0.067)	negligible (0.072)
Issue Solved	small (0.224)	not significant	negligible (0.025)
Issue Solving Ratio	small (0.178)	not significant	negligible (0.024)
Issue Solving Rate	small (0.189)	not significant	negligible (0.024)
Issue Contribution	small (0.298)	not significant	negligible (0.073)
Issue Contribution Rate	small (0.151)	negligible (0.08)	negligible (0.077)
Total Pull Requests	medium (0.405)	not significant	negligible (0.076)
PR Participated	large (0.531)	negligible (-0.106)	not significant
PR Reviewed	medium (0.413)	negligible (-0.105)	not significant
PR Merged	medium (0.385)	not significant	negligible (0.095)
PR Approval Ratio	small (-0.333)	negligible (0.072)	negligible (0.071)
PR Comments Received	small (0.189)	small (0.174)	negligible (0.085)
PR Contribution	large (0.492)	not significant	negligible (0.077)
PR Contribution Rate	small (-0.156)	negligible (0.074)	negligible (0.069)

Groups with no significant difference (i.e., null hypothesis is accepted) are indicated with 'not significant'. Groups that are significantly different examined by Mann-Whitney U test are indicated with Cliff's delta value and its interpretation (i.e., negligible, small, medium, or large). A positive value means group1 is greater than group2, and vice versa. Three columns from left to right correspond to the null hypothesis H_{01} , H_{02} , and H_{03} respectively.

D RQ2 and RQ3 Supplementary Result

Figure 11 shows an example of a newcomer being encouraged by Issue Discussants to become a contributor in TensorFlow. Issue Discussant A and Issue Discussant B are two existing contributors in TensorFlow with the workload composition pattern of Issue Discussant during the period from Oct 27th, 2017, to January 25th, 2018. In this period, a newcomer raises one’s first two issue reports (i.e., Issue 1 and Issue 2) on TensorFlow. Issue Discussant A actively engages in discussions for Issue 1 and Issue Discussant B participates in the discussions for both issues. In both issue reports, Issue Discussant B encourages the newcomer to make contributions. The newcomer submits one’s first pull request to address the observed issue (i.e., Pull Request 1), which is then reviewed and approved by Issue Discussant B. This newcomer eventually becomes a long-contributor to Tensorflow and remains active in the project for more than 4 years.

Figure 12a shows a commit with high commit importance (i.e., *commit centrality* = 0.018). This commit fixes the memory leak in the tensors, which is the fundamental functionality of the deep learning frameworks. The two files being modified in this commit contain the core and kernel functions, and changing these files can affect all computations happening within the framework. Figure 12b shows a commit with relatively low commit importance (i.e., *commit centrality* = $1.22e-24$). This commit makes changes to the demo code for a specific implementation of TensorFlow Lite on an Android device’s camera. The impact of such modifications is often limited to the specific functionalities and unlikely to cause large-scale disruptions.

Note that the figures in this appendix have been cropped to omit unimportant details. The original content can be found on TensorFlow GitHub repository³⁶³⁷³⁸³⁹⁴⁰.

³⁶<https://github.com/tensorflow/tensorflow/issues/15046>

³⁷<https://github.com/tensorflow/tensorflow/issues/15374>

³⁸<https://github.com/tensorflow/tensorflow/pull/15533>

³⁹<https://github.com/tensorflow/tensorflow/commit/2e8726c84a70ff962e11fac26472603aa92cc94e>

⁴⁰<https://github.com/tensorflow/tensorflow/commit/4b7511f4ecd6d0bd491ec557fe05fdfe731ecdae>

stop gradients for weights in tf.losses #15046

  closed this issue on Dec 1, 2017 • 8 comments

Newcomer

  commented on Dec 1, 2017 • edited

In the case that the weights given to `tf.losses.*` depend in some way on the model parameters, the derivative of that loss also calculated with respect to the weights.
(Stupid) minimal example:

```
import tensorflow as tf
x = tf.constant(0)
w = tf.get_variable(name="w", shape=(), initializer=tf.zeros_initializer())
L = tf.losses.mean_squared_error(x, x, weights=w)
tf.train.AdamOptimizer().compute_gradients(L)
```

results in

```
[(<tf.Tensor 'gradients/mean_squared_error/adj_grad/tuple/control_dependency_1:0' shape=() dtype=float32>,
<tf.Variable 'w:0' shape=() dtype=float32_ref>)]
```

I would expect the weights to be considered constant for the calculation of a loss. In case you agree with me, I can make a PR that adds `stop_gradient` around the weights parameter.

System information

- Have I written custom code (as opposed to using a stock example script provided in TensorFlow): No
- OS Platform and Distribution (e.g., Linux Ubuntu 16.04): N/A
- TensorFlow installed from (source or binary): N/A
- TensorFlow version (use command below): N/A
- Python version: N/A
- Bazel version (if compiling from source): N/A
- GCC/Compiler version (if compiling from source): N/A
- CUDA/cuDNN version: N/A
- GPU model and memory: N/A
- Exact command to reproduce: N/A

Issue Discussant A

 commented on Dec 1, 2017

Perhaps you'd better to use `tf.constant` for weight... instead of a variable.

  commented on Dec 4, 2017

In my case the weights are not constant, but calculated in the model. The above example is only supposed to be a minimal example to demonstrate the problem.

 commented on Dec 4, 2017 • edited

You can convert a variable to tensor when used, for example, `weights_ = weights + 0` or `weights_ = weights * 1`, and then pass `weights_` to `mean_squared_error`.

Issue Discussant B

  commented on Dec 6, 2017

Right, I also disagree that we should stop the gradients. I understand where you're coming from, but I think this would be confusing non-transparent behavior for people. Also, once you stop the gradient within the loss it's not possible to go back to previous behavior.

  closed this as `completed` on Dec 6, 2017

  commented on Dec 6, 2017

Then it would make sense to include this in the documentation, making this behaviour deliberate instead of a design accident.

  commented on Dec 6, 2017 via email

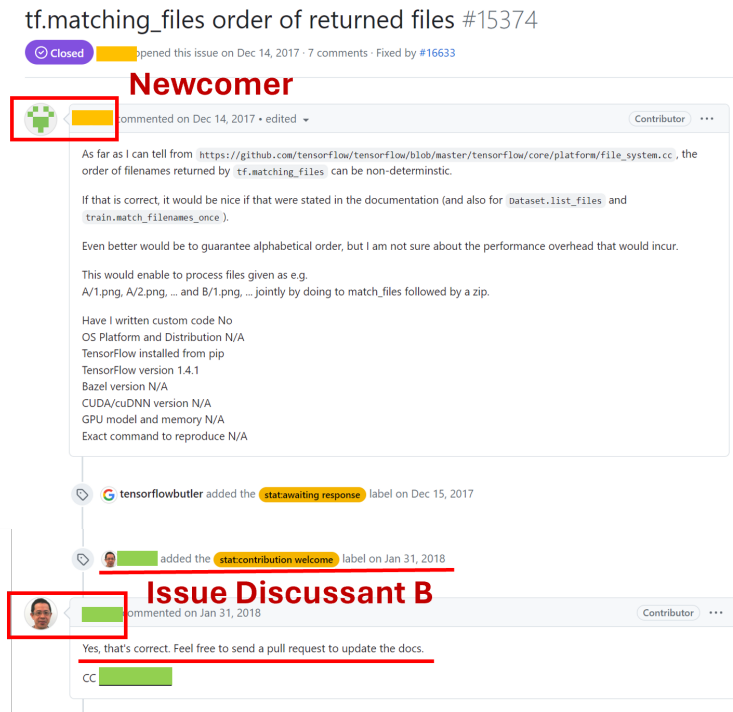
Sounds good. Feel free to send a PR

  mentioned this issue on Dec 20, 2017

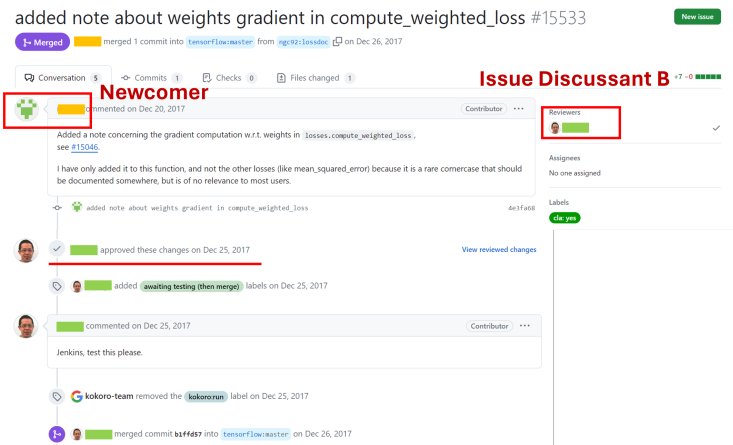
added note about weights gradient in `compute_weighted_loss` #15533

 Merged

(a) Issue 1



(b) Issue 2



(c) Pull Request 1

Fig. 11. A real-world example of newcomers encouraged by Issue Discussants to make contributions and become long-term contributors.

Commit

[fix mem leaks in tensor array and stack](#)

master (#1691)
v2.16.1 ... 0.12.0-rc0

committed on Apr 1, 2016

Showing 2 changed files with 16 additions and 1 deletion.

Filter changed files

- tensorflow/core/kernels
 - stack_ops.cc
 - tensor_array_ops.cc

```

@@ -26,6 +26,7 @@ limitations under the license.
26 26 #include "tensorflow/core/framework/tensor.h"
27 27 #include "tensorflow/core/framework/tensor_shape.h"
28 28 #include "tensorflow/core/framework/types.h"
29 + #include "tensorflow/core/lib/core/refcount.h"
29 30 #include "tensorflow/core/lib/core/errors.h"
30 31 #include "tensorflow/core/lib/gtl/map_util.h"
31 32 #include "tensorflow/core/platform/logging.h"

@@ -174,6 +175,7 @@ class StackPushOp : public AsyncOpKernel {
174 175 // Get the stack from the handle.
175 176 Stack* stack = nullptr;
176 177 OP_REQUIRES_OK(ctx, GetStack(ctx, &stack));
178 + core::ScopedUnref unref(stack);
177 179 OP_REQUIRES(ctx, ctx->input_dtype(1) == stack->ElemType(),
178 180 errors::InvalidArgument("Must have type ", stack->ElemType(),
179 181 " but got ", ctx->input_dtype(1)));

@@ -273,6 +275,7 @@ class StackPopOp : public AsyncOpKernel {
273 275 // Get the stack from the handle.
274 276 Stack* stack = nullptr;
275 277 OP_REQUIRES_OK(ctx, GetStack(ctx, &stack));
278 + core::ScopedUnref unref(stack);
276 279 // Pop the tensor. Transfer the tensor back to device if it was
277 280 // swapped out to CPU.
278 281

@@ -341,6 +344,7 @@ class StackCloseOp : public OpKernel {
341 344 void Compute(OpKernelContext* ctx) override {
342 345 Stack* stack = nullptr;
343 346 OP_REQUIRES_OK(ctx, GetStack(ctx, &stack));
347 + core::ScopedUnref unref(stack);
344 348 stack->Close();
345 349 }
346 350

```

(a) High commit centrality.

Commit

[Fix assets for the TF camera example.](#)

Mobile net model is downloaded from tf_http_archive("tf_mobilenet") rule and renaming the asset file in assets folder has no effect.

PiperOrigin-RevId: 188672531

master (#17651)
v2.16.1 ... tf-lite-v0.1.7

authored and tensorflow-gardener committed on Mar 11, 2018

Showing 1 changed file with 1 addition and 1 deletion.

```

...src/main/java/com/example/android/tflitecamerademo/ImageClassifierQuantizedMobileNet.java
@@ -46,7 +46,7 @@ protected String getModelPath() {
46 46
47 47 @Override
48 48 protected String getLabelPath() {
49 - return "labels_mobilenet_quant_v1_224.txt";
49 + return "labels.txt";
50 50 }
51 51
52 52 @Override

```

(b) Low commit centrality.

Manuscript submitted to ACM

Fig. 12. An example of commits with different levels of commit importance (commit centrality).

E RQ4 Supplementary Result

Table 14. Overall trend for Workload Composition, Work preference, and technical importance.

Profile	Core-Nighttime	Core-Daytime	Peripheral-Nighttime	Peripheral-Daytime
workload_sequence	↗(2.5%) ↘(0.6%)	↗(2.5%) ↘(0.6%)	↗(0.4%) ↘(0.1%)	↗(0.1%) ↘(0.2%)
binmed_entropy	↗(1.8%) ↘(4.0%)	↗(2.2%) ↘(5.9%)	↗(0.0%) ↘(0.3%)	↗(0.2%) ↘(0.6%)
c3(1)	↗(2.0%) ↘(2.6%)	↗(1.7%) ↘(4.6%)	↗(0.0%) ↘(0.1%)	↗(0.0%) ↘(0.2%)
c3(2)	↗(1.5%) ↘(2.1%)	↗(1.3%) ↘(4.2%)	↗(0.1%) ↘(0.0%)	↗(0.0%) ↘(0.1%)
c3(3)	↗(2.0%) ↘(2.3%)	↗(1.7%) ↘(4.3%)	↗(0.0%) ↘(0.1%)	↗(0.0%) ↘(0.1%)
number_cwt_peaks	↗(1.5%) ↘(2.8%)	↗(1.7%) ↘(5.4%)	↗(0.0%) ↘(0.3%)	↗(0.0%) ↘(0.4%)
longest_strike_above_mean	↗(1.1%) ↘(3.4%)	↗(1.1%) ↘(4.7%)	↗(0.0%) ↘(0.2%)	↗(0.0%) ↘(0.2%)
longest_strike_below_mean	↗(2.5%) ↘(1.7%)	↗(4.2%) ↘(1.5%)	↗(0.2%) ↘(0.2%)	↗(0.2%) ↘(0.1%)
diverse	↗(2.5%) ↘(2.3%)	↗(2.6%) ↘(4.0%)	↗(0.1%) ↘(0.3%)	↗(0.1%) ↘(0.2%)
balance	↗(4.1%) ↘(1.4%)	↗(6.8%) ↘(1.6%)	↗(0.3%) ↘(0.2%)	↗(0.3%) ↘(0.1%)
commit	↗(1.5%) ↘(4.9%)	↗(2.3%) ↘(7.2%)	↗(0.1%) ↘(0.1%)	↗(0.0%) ↘(0.1%)
issue	↗(1.1%) ↘(2.6%)	↗(1.3%) ↘(2.2%)	↗(0.0%) ↘(0.3%)	↗(0.0%) ↘(0.2%)
issue comment	↗(1.8%) ↘(3.2%)	↗(2.0%) ↘(5.2%)	↗(0.1%) ↘(0.2%)	↗(0.0%) ↘(0.5%)
pr comment	↗(2.6%) ↘(2.5%)	↗(2.6%) ↘(4.6%)	↗(0.1%) ↘(0.1%)	↗(0.0%) ↘(0.3%)
review	↗(2.8%) ↘(0.8%)	↗(3.7%) ↘(2.3%)	↗(0.1%) ↘(0.0%)	↗(0.1%) ↘(0.1%)
per commit centrality	↗(7.1%) ↘(9.9%)	↗(7.9%) ↘(12.4%)	↗(0.5%) ↘(0.9%)	↗(0.7%) ↘(0.7%)
period commit centrality	↗(1.4%) ↘(4.9%)	↗(1.6%) ↘(7.3%)	↗(0.0%) ↘(0.1%)	↗(0.0%) ↘(0.1%)

(%)↗ : The percentage of contributors reject the null hypothesis H_{013} and have an increasing trend in the given feature.

(%)↘ : The percentage of contributors reject the null hypothesis H_{014} and have a decreasing trend in the given feature.

Table 15. Early-middle and middle-late stage trend for workload composition, work preference, and technical importance.

Profile	Core-Nighttime			Core-Daytime			Peripheral-Nighttime			Peripheral-Daytime		
	Early-Middle	Middle-Late		Early-Middle	Middle-Late		Early-Middle	Middle-Late		Early-Middle	Middle-Late	
workload_sequence	↗(0.8%) ↘(0.6%) ↗(2.1%) ↘(0.4%) ↘(0.2%)	↗(2.6%) ↘(0.6%) ↗(2.1%) ↘(0.4%) ↘(0.2%)		↗(1.0%) ↘(0.1%) ↗(2.5%) ↘(0.3%) ↘(0.3%)	↗(3.2%) ↘(0.3%) ↗(3.2%) ↘(0.1%) ↘(0.1%)		↗(0.0%) ↘(0.0%) ↗(0.6%) ↘(0.0%) ↘(0.1%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↘(0.1%)		↗(0.0%) ↘(0.0%) ↗(0.1%) ↘(0.1%) ↘(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↘(0.0%)	
binmed_entropy	↗(2.1%) ↘(1.7%) ↗(1.3%) ↘(0.6%) ↗(0.4%)	↗(2.1%) ↘(0.8%) ↗(1.3%) ↘(0.4%) ↗(0.6%)		↗(3.9%) ↘(1.4%) ↗(3.1%) ↘(0.3%) ↗(1.9%)	↗(3.9%) ↘(1.4%) ↗(3.1%) ↘(0.3%) ↗(1.9%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	
c3(1)	↗(1.3%) ↘(0.6%) ↗(1.1%) ↘(0.8%) ↗(0.2%)	↗(1.3%) ↘(0.6%) ↗(1.1%) ↘(0.8%) ↗(0.2%)		↗(3.4%) ↘(1.2%) ↗(3.4%) ↘(0.7%) ↗(1.9%)	↗(3.4%) ↘(1.2%) ↗(3.4%) ↘(0.7%) ↗(1.9%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	
c3(2)	↗(2.5%) ↘(1.1%) ↗(2.3%) ↘(0.2%) ↗(0.0%)	↗(2.5%) ↘(1.1%) ↗(2.3%) ↘(0.2%) ↗(0.0%)		↗(3.6%) ↘(1.8%) ↗(3.6%) ↘(0.3%) ↗(2.3%)	↗(3.6%) ↘(1.8%) ↗(3.6%) ↘(0.3%) ↗(2.3%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	
c3(3)	↗(1.1%) ↘(1.3%) ↗(1.1%) ↘(0.0%) ↗(0.2%)	↗(1.1%) ↘(1.3%) ↗(1.1%) ↘(0.0%) ↗(0.2%)		↗(1.2%) ↘(1.4%) ↗(1.2%) ↘(0.2%) ↗(2.1%)	↗(1.2%) ↘(1.4%) ↗(1.2%) ↘(0.2%) ↗(2.1%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	
number_cwt_peaks	↗(1.1%) ↘(1.3%) ↗(1.1%) ↘(0.0%) ↗(0.2%)	↗(1.1%) ↘(1.3%) ↗(1.1%) ↘(0.0%) ↗(0.2%)		↗(1.2%) ↘(1.4%) ↗(1.2%) ↘(0.2%) ↗(2.1%)	↗(1.2%) ↘(1.4%) ↗(1.2%) ↘(0.2%) ↗(2.1%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	
longest_strike_above_mean	↗(1.3%) ↘(0.4%) ↗(2.1%) ↘(0.8%) ↗(0.6%)	↗(1.3%) ↘(0.4%) ↗(2.1%) ↘(0.8%) ↗(0.6%)		↗(2.1%) ↘(0.4%) ↗(3.8%) ↘(0.3%) ↗(1.9%)	↗(2.1%) ↘(0.4%) ↗(3.8%) ↘(0.3%) ↗(1.9%)		↗(0.3%) ↘(0.2%) ↗(0.3%) ↘(0.2%) ↗(0.0%)	↗(0.3%) ↘(0.2%) ↗(0.3%) ↘(0.2%) ↗(0.0%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	
diverse	↗(1.5%) ↘(0.4%) ↗(2.1%) ↘(0.8%) ↗(0.6%)	↗(1.5%) ↘(0.4%) ↗(2.1%) ↘(0.8%) ↗(0.6%)		↗(2.1%) ↘(0.4%) ↗(3.8%) ↘(0.3%) ↗(1.9%)	↗(2.1%) ↘(0.4%) ↗(3.8%) ↘(0.3%) ↗(1.9%)		↗(0.3%) ↘(0.2%) ↗(0.3%) ↘(0.2%) ↗(0.0%)	↗(0.3%) ↘(0.2%) ↗(0.3%) ↘(0.2%) ↗(0.0%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	
balance	↗(3.4%) ↘(2.1%) ↗(2.1%) ↘(0.9%) ↗(0.6%)	↗(3.4%) ↘(2.1%) ↗(2.1%) ↘(0.9%) ↗(0.6%)		↗(3.7%) ↘(3.8%) ↗(3.7%) ↘(0.1%) ↗(2.4%)	↗(3.7%) ↘(3.8%) ↗(3.7%) ↘(0.1%) ↗(2.4%)		↗(0.7%) ↘(0.5%) ↗(0.7%) ↘(0.5%) ↗(0.2%)	↗(0.7%) ↘(0.5%) ↗(0.7%) ↘(0.5%) ↗(0.2%)		↗(0.5%) ↘(0.6%) ↗(0.5%) ↘(0.6%) ↗(0.0%)	↗(0.5%) ↘(0.6%) ↗(0.5%) ↘(0.6%) ↗(0.0%)	
commit	↗(1.9%) ↘(2.6%) ↗(1.9%) ↘(0.2%) ↗(0.2%)	↗(1.9%) ↘(2.6%) ↗(1.9%) ↘(0.2%) ↗(0.2%)		↗(3.2%) ↘(3.2%) ↗(3.2%) ↘(0.8%) ↗(2.8%)	↗(3.2%) ↘(3.2%) ↗(3.2%) ↘(0.8%) ↗(2.8%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	
issue	↗(0.4%) ↘(1.7%) ↗(1.7%) ↘(0.6%) ↗(0.8%)	↗(0.4%) ↘(1.7%) ↗(1.7%) ↘(0.6%) ↗(0.8%)		↗(1.0%) ↘(0.3%) ↗(0.3%) ↘(0.2%) ↗(0.8%)	↗(1.0%) ↘(0.3%) ↗(0.3%) ↘(0.2%) ↗(0.8%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	
issue comment	↗(1.7%) ↘(2.3%) ↗(2.3%) ↘(0.2%) ↗(0.8%)	↗(1.7%) ↘(2.3%) ↗(2.3%) ↘(0.2%) ↗(0.8%)		↗(3.7%) ↘(2.2%) ↗(2.2%) ↘(0.3%) ↗(2.6%)	↗(3.7%) ↘(2.2%) ↗(2.2%) ↘(0.3%) ↗(2.6%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	
pr comment	↗(3.4%) ↘(0.9%) ↗(0.9%) ↘(0.8%) ↗(0.9%)	↗(3.4%) ↘(0.9%) ↗(0.9%) ↘(0.8%) ↗(0.9%)		↗(5.0%) ↘(1.4%) ↗(1.4%) ↘(0.3%) ↗(2.8%)	↗(5.0%) ↘(1.4%) ↗(1.4%) ↘(0.3%) ↗(2.8%)		↗(0.5%) ↘(0.1%) ↗(0.5%) ↘(0.1%) ↗(0.0%)	↗(0.5%) ↘(0.1%) ↗(0.5%) ↘(0.1%) ↗(0.0%)		↗(0.2%) ↘(0.4%) ↗(0.2%) ↘(0.4%) ↗(0.0%)	↗(0.2%) ↘(0.4%) ↗(0.2%) ↘(0.4%) ↗(0.0%)	
review	↗(2.8%) ↘(0.0%) ↗(3.9%) ↘(0.8%) ↗(4.8%)	↗(2.8%) ↘(0.0%) ↗(3.9%) ↘(0.8%) ↗(4.8%)		↗(5.5%) ↘(0.1%) ↗(4.9%) ↘(0.8%) ↗(8.1%)	↗(5.5%) ↘(0.1%) ↗(4.9%) ↘(0.8%) ↗(8.1%)		↗(0.1%) ↘(0.0%) ↗(0.1%) ↘(0.0%) ↗(0.1%)	↗(0.1%) ↘(0.0%) ↗(0.1%) ↘(0.0%) ↗(0.1%)		↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	↗(0.1%) ↘(0.1%) ↗(0.1%) ↘(0.1%) ↗(0.0%)	
commit centrality	↗(2.7%) ↘(0.4%) ↗(0.4%) ↘(0.0%) ↗(0.8%)	↗(2.7%) ↘(0.4%) ↗(0.4%) ↘(0.0%) ↗(0.8%)		↗(3.5%) ↘(2.5%) ↗(2.5%) ↘(0.4%) ↗(5.0%)	↗(3.5%) ↘(2.5%) ↗(2.5%) ↘(0.4%) ↗(5.0%)		↗(0.3%) ↘(0.0%) ↗(0.3%) ↘(0.0%) ↗(0.1%)	↗(0.3%) ↘(0.0%) ↗(0.3%) ↘(0.0%) ↗(0.1%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	
period centrality	↗(0.4%) ↘(0.9%) ↗(0.9%) ↘(0.0%) ↗(0.8%)	↗(0.4%) ↘(0.9%) ↗(0.9%) ↘(0.0%) ↗(0.8%)		↗(1.3%) ↘(2.5%) ↗(2.5%) ↘(0.4%) ↗(5.0%)	↗(1.3%) ↘(2.5%) ↗(2.5%) ↘(0.4%) ↗(5.0%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)		↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	↗(0.0%) ↘(0.0%) ↗(0.0%) ↘(0.0%) ↗(0.0%)	

(%) ↗ : The percentage of contributors reject the null hypothesis $H0_{13}$ and have an increasing trend in the given feature.

(%) ↘ : The percentage of contributors reject the null hypothesis $H0_{14}$ and have a decreasing trend in the given feature.